AppointmentsFrame.java

```
package petpulse;

import java.awt.*; import java.sql.*; import java.time.LocalDate; import javax.swing.*; import javax.swing.table.DefaultTableModel;

/**

    • Appointment Booking & Scheduling Module.
    • Full self-service CRUD (Book, Update, Soft-Archive) for both Client and Admin accounts.

*/ public class AppointmentsFrame extends BaseFrame { private final JTextField txtId = UITheme.field("(auto)"); private final JTextField txtOwner = UITheme.field(""); private final JTextField txtDate = UITheme.field(LocalDate.now().toString()); private final JTextField txtTime = UITheme.field("09:00"); private final JTextField txtSearch = UITheme.searchField("Search appointments...");

private final JComboBox<String> cbPet = UITheme.combo();
private final JComboBox<String> cbService = UITheme.combo("Check-up", "Vaccination", "Grooming", "Surgery", "Dental Cleaning", "Deworming");
private final JComboBox<String> cbVet = UITheme.combo("Dr. Grey", "Dr. House");
private final JComboBox<String> cbStatus = UITheme.combo("Scheduled", "Completed", "Cancelled");

private final DefaultTableModel model = UITheme.model("Appointment ID", "Pet Name", "Owner", "Date", "Time", "Service", "Veterinarian", "Status");
private final JTable table = new JTable(model);

public AppointmentsFrame() {
    super("Appointments", "Appointments", "Schedule clinic visits and keep track of every booking");
    boolean admin = Session.isAdmin();
    UITheme.lock(txtId);
    UITheme.lock(txtOwner);

    Db.bindPets(cbPet, txtOwner);

    JPanel form = UITheme.formCard(4);
    form.add(UITheme.box("Appointment ID", txtId));
    form.add(UITheme.box("Pet Name / ID", cbPet));
    form.add(UITheme.box("Owner Name", txtOwner));
    form.add(UITheme.box("Service", cbService));
    form.add(UITheme.box("Date (YYYY-MM-DD)", txtDate));
    form.add(UITheme.box("Time 24h (HH:MM)", txtTime));
    form.add(UITheme.box("Veterinarian", cbVet));
    form.add(UITheme.box("Status", cbStatus));

    JButton btnAdd = UITheme.createBtn(admin ? "Add" : "Book Appointment", UITheme.BRIGHT_TEAL);
    JButton btnUpd = UITheme.createBtn("Update", UITheme.STEEL_BLUE);
    JButton btnArc = UITheme.createBtn("Archive", UITheme.ARCHIVE_RED);
    JButton btnClr = UITheme.createBtn("Clear", UITheme.STEEL_BLUE);
    JPanel btns = new JPanel(new FlowLayout(FlowLayout.LEFT, 10, 0));
    btns.setOpaque(false);
    btns.add(btnAdd);
    btns.add(btnUpd); // Enabled for Client self-service updates
    btns.add(btnArc); // Enabled for Client soft-delete archiving
    btns.add(btnClr);

    JPanel bar = new JPanel(new BorderLayout());
    bar.setOpaque(false);
    bar.add(btns, BorderLayout.WEST);
    bar.add(txtSearch, BorderLayout.EAST);

    JPanel top = new JPanel(new BorderLayout(0, 12));
    top.setOpaque(false);
    top.add(form, BorderLayout.NORTH);
    top.add(bar, BorderLayout.SOUTH);

    JPanel root = new JPanel(new BorderLayout(0, 14));
    root.setOpaque(false);
    root.add(top, BorderLayout.NORTH);
    root.add(UITheme.tableCard(table), BorderLayout.CENTER);
    setBody(root);

    UITheme.attachSearch(txtSearch, table);
    if (!admin) cbStatus.setEnabled(false); // Only admin can switch status to Completed/Cancelled

    table.getSelectionModel().addListSelectionListener(e -> {
        if (e.getValueIsAdjusting()) return;
```

```
            int r = table.getSelectedRow();
            if (r < 0 || r >= table.getRowCount()) return;
            try {
                r = table.convertRowIndexToModel(r);
                txtId.setText(v(r, 0));
                Db.selectPet(cbPet, v(r, 1));
                txtOwner.setText(v(r, 2));
                txtDate.setText(v(r, 3));
                txtTime.setText(v(r, 4));
                cbService.setSelectedItem(v(r, 5));
                cbVet.setSelectedItem(v(r, 6));
                cbStatus.setSelectedItem(v(r, 7));
            } catch (Exception ex) { }
        });

        btnClr.addActionListener(e -> clearForm());
        btnAdd.addActionListener(e -> addAppointment());
        btnUpd.addActionListener(e -> updateAppointment());
        btnArc.addActionListener(e -> archiveAppointment());

        loadData();
        clearForm();
    }

    private String v(int r, int c) {
        Object val = model.getValueAt(r, c);
       return val == null ? "" : String.valueOf(val);
    }

    private void clearForm() {
        txtId.setText("(auto)");
        txtDate.setText(LocalDate.now().toString());
        txtTime.setText("09:00");
        cbService.setSelectedIndex(0);
        cbVet.setSelectedIndex(0);
        cbStatus.setSelectedIndex(0);

        Db.resetPetCombo(cbPet);
        if (!Session.isAdmin()) {
            txtOwner.setText(Session.getFullName());
        } else {
            txtOwner.setText("");
        }
        table.clearSelection();
    }

    private String extractPetName(Object item) {
        if (item == null) return "";
        String s = item.toString().trim();
        return s.contains(" - ") ? s.split(" - ", 2)[1].trim() : s;
    }

    private String check() {
        Object selectedPet = cbPet.getSelectedItem();
        if (selectedPet == null || selectedPet.toString().trim().isEmpty()) {
            return "Please select or type a registered Pet Name / ID first.";
        }
        return Validation.firstError(
            Validation.date(txtDate.getText(), "Date"),
            Validation.time(txtTime.getText())
        );
    }

    private boolean conflict(Connection c, String id) throws SQLException {
        String vet = cbVet.getSelectedItem() != null ? cbVet.getSelectedItem().toString() : "";
        try (PreparedStatement ps = c.prepareStatement(
                "SELECT COUNT(*) FROM appointments WHERE veterinarian=? AND appointment_date=? AND appointment_time=? AND status='Scheduled' AND appointment_id<>?")) {
            ps.setString(1, vet);
            ps.setString(2, txtDate.getText().trim());
            ps.setString(3, txtTime.getText().trim());
            ps.setString(4, id);
            try (ResultSet rs = ps.executeQuery()) {
                return rs.next() && rs.getInt(1) > 0;
            }
        }
    }
```

```
private void addAppointment() {
    String err = check();
    if (err != null) { UITheme.toast(this, err, false); return; }

    String petName = extractPetName(cbPet.getSelectedItem());
    String ownerName = Session.isAdmin() ? txtOwner.getText().trim() : Session.getFullName();
    String service = cbService.getSelectedItem().toString();
    String vet = cbVet.getSelectedItem().toString();
    String status = cbStatus.getSelectedItem().toString();

    try (Connection c = DBConnection.getConnection()) {
        String id = Db.nextId(c, "Appointments", "APT-");
        if ("Scheduled".equals(status) && conflict(c, id)) {
            UITheme.toast(this, vet + " is already booked at that date and 24h time.", false);
            return;
        }
        try (PreparedStatement ps = c.prepareStatement(
                "INSERT INTO appointments (appointment_id, pet_name, owner_name, appointment_date, appointment_time, service, veterinarian, status) VALUES (?,?,?,?,?,?,?,?)")) {
            ps.setString(1, id);
            ps.setString(2, petName);
            ps.setString(3, ownerName);
            ps.setString(4, txtDate.getText().trim());
            ps.setString(5, txtTime.getText().trim());
            ps.setString(6, service);
            ps.setString(7, vet);
            ps.setString(8, status);
            ps.executeUpdate();
        }

        Db.syncAllAppointmentsToPayments();
        DBConnection.log("Appointments", "Add", id + " - " + petName + " (" + ownerName + ")");
        UITheme.toast(this, "Appointment scheduled & Payment invoice auto-generated!", true);
        clearForm();
        loadData();
    } catch (Exception ex) { UITheme.showError(this, ex); }
}

private void updateAppointment() {
    if ("(auto)".equals(txtId.getText())) { UITheme.toast(this, "Select an appointment from the table first.", false); return; }
    String err = check();
    if (err != null) { UITheme.toast(this, err, false); return; }

    String petName = extractPetName(cbPet.getSelectedItem());
    String ownerName = txtOwner.getText().trim();
    String service = cbService.getSelectedItem().toString();
    String vet = cbVet.getSelectedItem().toString();
    String status = cbStatus.getSelectedItem().toString();

    try (Connection c = DBConnection.getConnection()) {
        if ("Scheduled".equals(status) && conflict(c, txtId.getText())) {
            UITheme.toast(this, vet + " is already booked at that date and 24h time.", false);
            return;
        }
        try (PreparedStatement ps = c.prepareStatement(
                "UPDATE appointments SET pet_name=?, owner_name=?, appointment_date=?, appointment_time=?, service=?, veterinarian=?, status=? WHERE appointment_id=?")) {
            ps.setString(1, petName);
            ps.setString(2, ownerName);
            ps.setString(3, txtDate.getText().trim());
            ps.setString(4, txtTime.getText().trim());
            ps.setString(5, service);
            ps.setString(6, vet);
            ps.setString(7, status);
            ps.setString(8, txtId.getText());
            ps.executeUpdate();
        }

        Db.syncAllAppointmentsToPayments();
        DBConnection.log("Appointments", "Update", txtId.getText() + " updated");
        UITheme.toast(this, "Appointment & Payment synced.", true);
        clearForm();
        loadData();
    } catch (Exception ex) { UITheme.showError(this, ex); }
}

private void archiveAppointment() {
```

```
    if ("(auto)".equals(txtId.getText())) { UITheme.toast(this, "Select an appointment from the table first.", false); return; }
    if (JOptionPane.showConfirmDialog(this, "Move " + txtId.getText() + " to the archive?", "Archive Appointment", JOptionPane.YES_NO_OPTION) != JOptionPane.YES_OPTION) return;
    try {
        Db.archiveRecord("Appointments", txtId.getText(), extractPetName(cbPet.getSelectedItem()) + " (" + cbService.getSelectedItem() + ")");
        DBConnection.log("Appointments", "Archive", txtId.getText() + " archived");
        UITheme.toast(this, "Appointment moved to archive.", true);
        clearForm();
        loadData();
    } catch (Exception ex) { UITheme.showError(this, ex); }
}


private void loadData() {
    try {
        Db.syncAllAppointmentsToPayments();
        if (Session.isAdmin()) {
            Db.fill(model, "SELECT appointment_id, pet_name, owner_name, appointment_date, appointment_time, service, veterinarian, status FROM appointments ORDER BY appointment_date DESC,
appointment_time", 8);
        } else {
            String owner = Session.getFullName();
            model.setRowCount(0);
            try (Connection c = DBConnection.getConnection();
                 PreparedStatement ps = c.prepareStatement(
                   "SELECT appointment_id, pet_name, owner_name, appointment_date, appointment_time, service, veterinarian, status FROM appointments WHERE owner_name=? ORDER BY appointment_date
DESC, appointment_time")) {
                ps.setString(1, owner);
                try (ResultSet rs = ps.executeQuery()) {
                    while (rs.next()) {
                        Object[] row = new Object[8];
                        for (int i = 0; i < 8; i++) {
                            String val = rs.getString(i + 1);
                            row[i] = val == null ? "" : val;
                        }
                        model.addRow(row);
                    }
                }
            }
        }
    } catch (Exception e) {
        e.printStackTrace();
        UITheme.toast(this, "Could not load appointments. Is MySQL running?", false);
    }
}



}
```

ArchiveFrame.java ×

```
package petpulse;

import java.awt.; import java.sql.; import javax.swing.*; import javax.swing.table.DefaultTableModel;

/** Soft-delete vault: Restore, Delete Permanently, Clear Archive (Admin only). */ public class ArchiveFrame extends BaseFrame { private final DefaultTableModel model = UITheme.model("Archive ID", "Module",
"Record ID", "Summary", "Archived At", "Full Data"); private final JTable table = new JTable(model); private final JTextField txtSearch = UITheme.searchField("Search archive…");

public ArchiveFrame() {
    super("Archive", "Archive", "Archived records are safe here - restore them any time");


   JButton btnRestore = UITheme.createBtn("Restore", UITheme.BRIGHT_TEAL);
 JButton btnDel = UITheme.createBtn("Delete Permanently", UITheme.ARCHIVE_RED);
    JButton btnClear = UITheme.createBtn("Clear Archive", UITheme.STEEL_BLUE);
    JPanel btns = new JPanel(new FlowLayout(FlowLayout.LEFT, 10, 0));
    btns.setOpaque(false);
    btns.add(btnRestore);
    btns.add(btnDel);
    btns.add(btnClear);
    JPanel bar = new JPanel(new BorderLayout());
    bar.setOpaque(false);
    bar.add(btns, BorderLayout.WEST);
    bar.add(txtSearch, BorderLayout.EAST);


    JPanel root = new JPanel(new BorderLayout(0, 14));
    root.setOpaque(false);
    root.add(bar, BorderLayout.NORTH);
    root.add(UITheme.tableCard(table), BorderLayout.CENTER);
    setBody(root);


    table.removeColumn(table.getColumnModel().getColumn(5)); // hide raw data column
```

```
    UITheme.attachSearch(txtSearch, table);

    btnRestore.addActionListener(e -> {
        int r = selectedModelRow();
        if (r < 0) return;
        String mod = v(r, 1), rec = v(r, 2);
        try {
            Db.restoreRecord(Integer.parseInt(v(r, 0)), mod, v(r, 5));
        } catch (Exception ex) {
            UITheme.showError(this, ex);
            return;
        }
        DBConnection.log("Archive", "Restore", mod + " - " + rec);
        UITheme.toast(this, rec + " restored to " + mod + ".", true);
        loadArchive();
    });
    btnDel.addActionListener(e -> {
        int r = selectedModelRow();
        if (r < 0) return;
        if (JOptionPane.showConfirmDialog(this, "Permanently delete " + v(r, 2) + "?\nThis cannot be undone.", "Delete Permanently", JOptionPane.YES_NO_OPTION) != JOptionPane.YES_OPTION) return;
        try (Connection c = DBConnection.getConnection();
             PreparedStatement ps = c.prepareStatement("DELETE FROM archive WHERE archive_id=?")) {
            ps.setInt(1, Integer.parseInt(v(r, 0)));
            ps.executeUpdate();
        } catch (Exception ex) { UITheme.showError(this, ex); return; }
        DBConnection.log("Archive", "Delete", "Permanently deleted " + v(r, 2));
        UITheme.toast(this, "Record deleted permanently.", true);
        loadArchive();
    });
    btnClear.addActionListener(e -> {
        if (model.getRowCount() == 0) { UITheme.toast(this, "The archive is already empty.", false); return; }
        if (JOptionPane.showConfirmDialog(this, "Permanently delete EVERYTHING in the archive?\nThis cannot be undone.", "Clear Archive", JOptionPane.YES_NO_OPTION) != JOptionPane.YES_OPTION)
return;
        try (Connection c = DBConnection.getConnection(); Statement s = c.createStatement()) {
            s.executeUpdate("DELETE FROM archive");
        } catch (Exception ex) { UITheme.showError(this, ex); return; }
        DBConnection.log("Archive", "Clear", "Archive cleared");
        UITheme.toast(this, "Archive cleared.", true);
        loadArchive();
    });
    loadArchive();
}

private String v(int r, int c) { return String.valueOf(model.getValueAt(r, c)); }

private int selectedModelRow() {
    int r = table.getSelectedRow();
    if (r < 0) {
        UITheme.toast(this, "Select a record in the table first.", false);
        return -1;
    }
    return table.convertRowIndexToModel(r);
}

private void loadArchive() {
    try {
        Db.fill(model, "SELECT archive_id, module_name, record_id, summary, DATE_FORMAT(archived_at,'%Y-%m-%d %H:%i:%s'), full_data FROM archive ORDER BY archive_id DESC", 6);
    } catch (Exception e) {
        e.printStackTrace();
        UITheme.toast(this, "Could not load archive. Is MySQL running?", false);
    }
}

}
```

BaseFrame.java

```
package petpulse;

import java.awt.*; import java.awt.event.*; import java.text.SimpleDateFormat; import java.util.Date; import javax.swing.*; import javax.swing.border.EmptyBorder;

/**

    - Common window shell: sidebar + page header + content area with a fade-in animation.
    - Every module frame extends this and only builds its own content (setBody).

*/ public abstract class BaseFrame extends JFrame { public final String module; private final FadePanel holder = new FadePanel(); private final JLabel lblClock = new JLabel(); private final Timer clock;
```

```
protected BaseFrame(String module, String title, String subtitle) {
    super("PetPulse - " + module);
   this.module = module;
    setIconImage(Logo.image(64));
    setSize(1280, 760);
    setMinimumSize(new Dimension(1100, 680));
    setLocationRelativeTo(null);
    setDefaultCloseOperation(DO_NOTHING_ON_CLOSE);

    Container c = getContentPane();
    c.setLayout(new BorderLayout());
    c.add(Nav.createSidebar(this, module), BorderLayout.WEST);

    JPanel right = new JPanel(new BorderLayout(0, 18));
    right.setBackground(UITheme.COOL_OFF_WHITE);
    right.setBorder(new EmptyBorder(24, 28, 24, 28));

    JPanel head = new JPanel(new BorderLayout());
    head.setOpaque(false);
    JPanel titles = new JPanel();
    titles.setOpaque(false);
    titles.setLayout(new BoxLayout(titles, BoxLayout.Y_AXIS));
    titles.add(UITheme.label(title, UITheme.FONT_TITLE, UITheme.NIGHT_NAVY));
    titles.add(Box.createVerticalStrut(2));
    titles.add(UITheme.label(subtitle, UITheme.FONT_REG, UITheme.STEEL_BLUE));
    lblClock.setFont(UITheme.FONT_BOLD);
    lblClock.setForeground(UITheme.STEEL_BLUE);
    head.add(titles, BorderLayout.WEST);
    head.add(lblClock, BorderLayout.EAST);

    right.add(head, BorderLayout.NORTH);
    right.add(holder, BorderLayout.CENTER);
    c.add(right, BorderLayout.CENTER);

    SimpleDateFormat fmt = new SimpleDateFormat("EEEE, MMMM d, yyyy  |  hh:mm:ss a");
    lblClock.setText(fmt.format(new Date()));
    clock = new Timer(1000, e -> lblClock.setText(fmt.format(new Date())));
    clock.start();

    addWindowListener(new WindowAdapter() {
        @Override public void windowOpened(WindowEvent e) { holder.fade(); }
        @Override public void windowClosed(WindowEvent e) { clock.stop(); }
        @Override public void windowClosing(WindowEvent e) {
            if (JOptionPane.showConfirmDialog(BaseFrame.this, "Exit PetPulse?", "Exit", JOptionPane.YES_NO_OPTION) == JOptionPane.YES_OPTION) {
                System.exit(0);
            }
        }
    });
}

/** Subclasses put their screen here. */
protected void setBody(JComponent body) {
    holder.add(body, BorderLayout.CENTER);
}

/** Content area that fades in when the window opens. */
private static class FadePanel extends JPanel {
    private float alpha = 1f;

    FadePanel() {
        super(new BorderLayout());
        setBackground(UITheme.COOL_OFF_WHITE);
        // safety: never stay hidden even if the open event is missed
        Timer safety = new Timer(1800, e -> { alpha = 0f; repaint(); });
        safety.setRepeats(false);
        safety.start();
    }

    void fade() {
        alpha = 1f;
        Timer t = new Timer(16, null);
        t.addActionListener(e -> {
            alpha -= 0.08f;
            if (alpha <= 0f) { alpha = 0f; t.stop(); }
            repaint();
```

```
        });
        t.start();
    }

    @Override public void paint(Graphics g) {
        super.paint(g);
        if (alpha > 0f) {
            Graphics2D g2 = (Graphics2D) g.create();
            g2.setComposite(AlphaComposite.SrcOver.derive(Math.min(1f, alpha)));
            g2.setColor(getBackground());
            g2.fillRect(0, 0, getWidth(), getHeight());
            g2.dispose();
        }
    }
}



}
```




```
package petpulse;

import java.sql.*; import java.time.LocalDate; import javax.swing.JOptionPane;

/**

    • Database connection + automatic setup.
    • On startup it creates the database, all tables and sample data if they do not exist yet,
    • so the system works on any computer that has MySQL running.

*/ public class DBConnection {

// ---- CHANGE THESE IF YOUR MYSQL IS DIFFERENT ----
private static final String HOST = "localhost:3306";
private static final String DB = "petpulse_db";
private static final String USER = "root";
private static final String PASSWORD = "";
// -------------------------------------------------
private static final String PARAMS = "?useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC&useUnicode=true&characterEncoding=UTF-8";

static {
    try {Class.forName("com.mysql.cj.jdbc.Driver");
    } catch (ClassNotFoundException e) {
        try { Class.forName("com.mysql.jdbc.Driver"); } catch (ClassNotFoundException ignored) { }
    }
}

public static Connection getConnection() throws SQLException {
    return DriverManager.getConnection("jdbc:mysql://" + HOST + "/" + DB + PARAMS, USER, PASSWORD);
}

/** Creates database, tables and first-run sample data. Returns false if MySQL cannot be reached. */
public static boolean init() {
    try {
        try (Connection c = DriverManager.getConnection("jdbc:mysql://" + HOST + "/" + PARAMS, USER, PASSWORD);
             Statement s = c.createStatement()) {
            s.executeUpdate("CREATE DATABASE IF NOT EXISTS " + DB + " CHARACTER SET utf8mb4");
        }
        try (Connection c = getConnection(); Statement s = c.createStatement()) {
            createTables(s);
            seed(c);
        }
        return true;
    } catch (Exception e) {
        e.printStackTrace();
        JOptionPane.showMessageDialog(null,
            "PetPulse could not connect to MySQL.\n\n"
            + "1. Start MySQL (XAMPP Control Panel > MySQL > Start)\n"
            + "2. Add the MySQL JDBC driver (mysql-connector-j) to the project Libraries\n"
            + "3. Check the USER and PASSWORD at the top of DBConnection.java\n\n"
            + "Details: " + e.getMessage(),
            "PetPulse - Database Error", JOptionPane.ERROR_MESSAGE);
        return false;
    }
}
```

```
private static void createTables(Statement s) throws SQLException {
    s.executeUpdate("CREATE TABLE IF NOT EXISTS pets (pet_id VARCHAR(20) PRIMARY KEY, pet_name VARCHAR(100) NOT NULL, species VARCHAR(30), breed VARCHAR(100), age INT DEFAULT 0,
gender VARCHAR(10), owner_name VARCHAR(100), contact_number VARCHAR(30), health_status VARCHAR(30), vaccination_status VARCHAR(40))");
    s.executeUpdate("CREATE TABLE IF NOT EXISTS appointments (appointment_id VARCHAR(20) PRIMARY KEY, pet_name VARCHAR(100), owner_name VARCHAR(100), appointment_date DATE,
appointment_time VARCHAR(10), service VARCHAR(40), veterinarian VARCHAR(60), status VARCHAR(20))");
    s.executeUpdate("CREATE TABLE IF NOT EXISTS vaccinations (vaccination_id VARCHAR(20) PRIMARY KEY, pet_name VARCHAR(100), vaccine_name VARCHAR(60), date_given DATE, next_due_date
DATE, dosage VARCHAR(30), veterinarian VARCHAR(60), status VARCHAR(20))");
    s.executeUpdate("CREATE TABLE IF NOT EXISTS payments (payment_id VARCHAR(20) PRIMARY KEY, pet_name VARCHAR(100), owner_name VARCHAR(100), service VARCHAR(40), amount
DECIMAL(10,2), payment_method VARCHAR(20), payment_date DATE, status VARCHAR(20))");
    s.executeUpdate("CREATE TABLE IF NOT EXISTS transaction_history (log_id INT AUTO_INCREMENT PRIMARY KEY, timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP, module_name
VARCHAR(50), action_type VARCHAR(30), details VARCHAR(255))");
    s.executeUpdate("CREATE TABLE IF NOT EXISTS archive (archive_id INT AUTO_INCREMENT PRIMARY KEY, module_name VARCHAR(50), record_id VARCHAR(20), summary VARCHAR(255),
archived_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP, full_data TEXT)");
    s.executeUpdate("CREATE TABLE IF NOT EXISTS app_users (user_id INT AUTO_INCREMENT PRIMARY KEY, username VARCHAR(50) UNIQUE, password_hash CHAR(64), role VARCHAR(10),
full_name VARCHAR(100))");
}


private static int count(Connection c, String table) throws SQLException {
    try (Statement s = c.createStatement(); ResultSet rs = s.executeQuery("SELECT COUNT(*) FROM " + table)) {
        rs.next();
        return rs.getInt(1);
    }
}


private static void exec(Connection c, String sql, Object... args) throws SQLException {
    try (PreparedStatement ps = c.prepareStatement(sql)) {
        for (int i = 0; i < args.length; i++) ps.setObject(i + 1, args[i]);
        ps.executeUpdate();
    }
}


/** First run only: default accounts + sample records so the screens are not empty. */
private static void seed(Connection c) throws SQLException {
    if (count(c, "app_users") > 0) return;
    String ins = "INSERT INTO app_users (username, password_hash, role, full_name) VALUES (?,?,?,?)";
    exec(c, ins, "admin", Session.hash("admin123"), "Admin", "Administrator");
    exec(c, ins, "client", Session.hash("client123"), "Client", "Maria Dela Cruz");

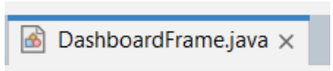

    LocalDate today = LocalDate.now();
    if (count(c, "pets") == 0) {
        String p = "INSERT INTO pets VALUES (?,?,?,?,?,?,?,?,?,?)";
        exec(c, p, "PET-001", "Bantay", "Dog", "Aspin", 3, "Male", "Maria Dela Cruz", "09171234567", "Healthy", "Vaccinated");
        exec(c, p, "PET-002", "Mingming", "Cat", "Puspin", 2, "Female", "Juan Reyes", "09281234567", "Healthy", "Partially Vaccinated");
        exec(c, p, "PET-003", "Coco", "Dog", "Shih Tzu", 5, "Female", "Ana Lopez", "09391234567", "Under Treatment", "Not Vaccinated");
        exec(c, p, "PET-004", "Goldy", "Dog", "Golden Retriever", 2, "Female", "April", "09123456789", "Healthy", "Vaccinated");
    }
    if (count(c, "appointments") == 0) {
        String a = "INSERT INTO appointments VALUES (?,?,?,?,?,?,?,?)";
        exec(c, a, "APT-001", "Bantay", "Maria Dela Cruz", today.toString(), "09:00", "Check-up", "Dr. Santos", "Scheduled");
        exec(c, a, "APT-002", "Coco", "Ana Lopez", today.plusDays(1).toString(), "10:30", "Vaccination", "Dr. Reyes", "Scheduled");
    }
    if (count(c, "vaccinations") == 0) {
        String v = "INSERT INTO vaccinations VALUES (?,?,?,?,?,?,?,?)";
        exec(c, v, "VAC-001", "Bantay", "Rabies", today.minusDays(9).toString(), today.plusYears(1).minusDays(9).toString(), "1 ml", "Dr. Santos", "Done");
        exec(c, v, "VAC-002", "Coco", "DHPP", today.minusDays(9).toString(), today.plusDays(5).toString(), "1 ml", "Dr. Reyes", "Pending");
    }
    if (count(c, "payments") == 0) {
        String y = "INSERT INTO payments VALUES (?,?,?,?,?,?,?,?)";
        exec(c, y, "PAY-001", "Bantay", "Maria Dela Cruz", "Check-up", 500.00, "Cash", today.minusDays(9).toString(), "Paid");
        exec(c, y, "PAY-002", "Mingming", "Juan Reyes", "Grooming", 350.00, "GCash", today.minusDays(9).toString(), "Paid");
    }
}


/** Writes one line in the Transaction History audit log. Never throws. */
public static void log(String module, String action, String details) {
    String who = Session.getUsername().isEmpty() ? "" : "[" + Session.getUsername() + "] ";
    String text = who + details;
    if (text.length() > 250) text = text.substring(0, 250);
    try (Connection c = getConnection();
         PreparedStatement ps = c.prepareStatement("INSERT INTO transaction_history (module_name, action_type, details) VALUES (?,?,?)")) {
        ps.setString(1, module);
        ps.setString(2, action);
        ps.setString(3, text);
        ps.executeUpdate();
    } catch (SQLException e) {
        e.printStackTrace();
```

```
    }
}



}
```




```
package petpulse;

import java.awt.; import java.awt.geom.RoundRectangle2D; import java.sql.*; import java.time.LocalDate; import javax.swing.*; import javax.swing.border.EmptyBorder; import javax.swing.table.DefaultTableModel;

/**

    • Dashboard Module.
    • Admin: View clinic-wide performance metrics, revenue, and master schedules.
    • Client: Strict row-level privacy showing ONLY personal pets, personal visits, payments, and due vaccinations.

*/ public class DashboardFrame extends BaseFrame { private final StatCard cPets; private final StatCard cApts; private final StatCard cPay; private final StatCard cVac; private final DefaultTableModel mApt = UITheme.model("Date", "Time", "Pet", "Service", "Veterinarian"); private final DefaultTableModel mVac = UITheme.model("Pet", "Vaccine", "Next Due", "Veterinarian");

public DashboardFrame() {
  super("Dashboard", "Dashboard Overview",
      Session.isAdmin()
        ? "Welcome back, " + (Session.getFullName().isEmpty() ? "Administrator" : Session.getFullName()) + " - here is your clinic at a glance"
        : "Welcome back, " + (Session.getFullName().isEmpty() ? "Client" : Session.getFullName()) + " - here is your personal pet overview");


  boolean admin = Session.isAdmin();
  cPets = new StatCard(admin ? "Total Pets" : "My Registered Pets", UITheme.BRIGHT_TEAL, false);
  cApts = new StatCard(admin ? "Today's Appointments" : "My Today's Visits", UITheme.BLUE, false);
  cPay = new StatCard(admin ? "Total Payments (Paid)" : "My Total Payments", UITheme.AMBER, true);
  cVac = new StatCard(admin ? "Pending Vaccinations" : "My Due Vaccinations", UITheme.ARCHIVE_RED, false);


  JPanel cards = new JPanel(new GridLayout(1, 4, 18, 0));
  cards.setOpaque(false);
  cards.setPreferredSize(new Dimension(100, 128));
  cards.add(cPets);
  cards.add(cApts);
  cards.add(cPay);
  cards.add(cVac);


  JPanel lower = new JPanel(new GridLayout(1, 2, 18, 0));
  lower.setOpaque(false);
  lower.add(UITheme.sectionCard(admin ? "Upcoming Appointments" : "My Upcoming Visits", new JTable(mApt)));
  lower.add(UITheme.sectionCard(admin ? "Pending Vaccinations" : "My Due Vaccinations", new JTable(mVac)));


  JPanel root = new JPanel(new BorderLayout(0, 18));
  root.setOpaque(false);
  root.add(cards, BorderLayout.NORTH);
  root.add(lower, BorderLayout.CENTER);
  setBody(root);


  loadStats();
}


private void loadStats() {
  String today = LocalDate.now().toString();
  boolean admin = Session.isAdmin();
  String clientName = Session.getFullName();


  try (Connection c = DBConnection.getConnection()) {
    if (admin) {
      // Admin View: Clinic-wide metrics
      try (Statement s = c.createStatement()) {
        cPets.countTo(one(s, "SELECT COUNT(*) FROM pets"));
        cApts.countTo(one(s, "SELECT COUNT(*) FROM appointments WHERE appointment_date = '" + today + "' AND status='Scheduled'"));
        cPay.countTo(one(s, "SELECT IFNULL(SUM(amount),0) FROM payments WHERE status='Paid'"));
        cVac.countTo(one(s, "SELECT COUNT(*) FROM vaccinations WHERE status='Pending'"));


        Db.fill(mApt, "SELECT appointment_date, appointment_time, pet_name, service, veterinarian FROM appointments "
          + "WHERE status='Scheduled' AND appointment_date >= '" + today + "' ORDER BY appointment_date, appointment_time LIMIT 8", 5);
        Db.fill(mVac, "SELECT pet_name, vaccine_name, next_due_date, veterinarian FROM vaccinations "
          + "WHERE status='Pending' ORDER BY next_due_date LIMIT 8", 4);
      }
    } else {
```

```
            // Client View: Strict Data Privacy Isolation (WHERE owner_name = ?)
            cPets.countTo(countPrepared(c, "SELECT COUNT(*) FROM pets WHERE owner_name=?", clientName));
            cApts.countTo(countPrepared(c, "SELECT COUNT(*) FROM appointments WHERE owner_name=? AND appointment_date=? AND status='Scheduled'", clientName, today));
            cPay.countTo(countPrepared(c, "SELECT IFNULL(SUM(amount),0) FROM payments WHERE owner_name=? AND status='Paid'", clientName));
            cVac.countTo(countPrepared(c, "SELECT COUNT(*) FROM vaccinations v JOIN pets p ON v.pet_name = p.pet_name WHERE p.owner_name=? AND v.status='Pending'", clientName));

            // Populate Client's Personal Upcoming Appointments
            mApt.setRowCount(0);
            try (PreparedStatement ps = c.prepareStatement(
                    "SELECT appointment_date, appointment_time, pet_name, service, veterinarian FROM appointments "
                  + "WHERE owner_name=? AND status='Scheduled' AND appointment_date >= ? ORDER BY appointment_date, appointment_time LIMIT 8")) {
                ps.setString(1, clientName);
                ps.setString(2, today);
                try (ResultSet rs = ps.executeQuery()) {
                    while (rs.next()) {
                        mApt.addRow(new Object[]{rs.getString(1), rs.getString(2), rs.getString(3), rs.getString(4), rs.getString(5)});
                    }
                }
            }

            // Populate Client's Personal Pending Vaccinations
            mVac.setRowCount(0);
            try (PreparedStatement ps = c.prepareStatement(
                    "SELECT v.pet_name, v.vaccine_name, v.next_due_date, v.veterinarian FROM vaccinations v JOIN pets p ON v.pet_name = p.pet_name "
                  + "WHERE p.owner_name=? AND v.status='Pending' ORDER BY v.next_due_date LIMIT 8")) {
                ps.setString(1, clientName);
                try (ResultSet rs = ps.executeQuery()) {
                    while (rs.next()) {
                        mVac.addRow(new Object[]{rs.getString(1), rs.getString(2), rs.getString(3), rs.getString(4)});
                    }
                }
            }
        }
    } catch (Exception e) {
        e.printStackTrace();
        UITheme.toast(this, "Could not load statistics. Is MySQL running?", false);
    }
}

private double one(Statement s, String sql) throws SQLException {
    try (ResultSet rs = s.executeQuery(sql)) {
        return rs.next() ? rs.getDouble(1) : 0;
    }
}

private double countPrepared(Connection c, String sql, String... params) throws SQLException {
    try (PreparedStatement ps = c.prepareStatement(sql)) {
        for (int i = 0; i < params.length; i++) {
            ps.setString(i + 1, params[i]);
        }
        try (ResultSet rs = ps.executeQuery()) {
            return rs.next() ? rs.getDouble(1) : 0;
        }
    }
}

/** Metric card with ultra-smooth 60 FPS quintic ease-out animation. */
private static class StatCard extends UITheme.RoundPanel {
    private final JLabel value = new JLabel();
    private final boolean money;
    private final Color accent;
    private Timer anim;

    StatCard(String title, Color accent, boolean money) {
        super(18, UITheme.CARD_BG);
        this.accent = accent;
        this.money = money;
        setLayout(new BorderLayout(0, 6));
        setBorder(new EmptyBorder(20, 26, 16, 18));
        value.setFont(new Font("Segoe UI", Font.BOLD, 34));
        value.setForeground(UITheme.NIGHT_NAVY);
        value.setText(fmt(0));
        add(UITheme.label(title, UITheme.FONT_BOLD, UITheme.STEEL_BLUE), BorderLayout.NORTH);
        add(value, BorderLayout.CENTER);
    }

    private String fmt(double v) {
```

```
        return money ? String.format("PHP %,.2f", v) : String.valueOf(Math.round(v));
    }

    void countTo(double target) {
        if (anim != null) anim.stop();
        if (target == 0) {
            value.setText(fmt(0));
            return;
        }

        final int[] step = {0};
        final int steps = 30; // 30 steps @ 16ms = ~480ms smooth duration
        anim = new Timer(16, null); // 16ms tick rate = 60 FPS refresh
        anim.addActionListener(e -> {
            step[0]++;
            double progress = Math.min(1.0, step[0] / (double) steps);

            // Exponential Quintic Ease-Out Motion: 1 - (1 - progress)^5
            double easeOut = 1.0 - Math.pow(1.0 - progress, 5);
            value.setText(fmt(target * easeOut));

            if (step[0] >= steps) {
                value.setText(fmt(target));
                ((Timer) e.getSource()).stop();
            }
        });
        anim.start();
    }

    @Override protected void paintChildren(Graphics g) {
        super.paintChildren(g);
        Graphics2D g2 = (Graphics2D) g.create();
        g2.setClip(new RoundRectangle2D.Float(0, 0, getWidth(), getHeight(), 18, 18));
        g2.setColor(accent);
        g2.fillRect(0, 0, 7, getHeight());
        g2.dispose();
    }
}



}
```

Db.java

```
package petpulse;

import java.awt.event.ActionListener; import java.math.BigDecimal; import java.math.RoundingMode; import java.sql.*; import java.util.*; import javax.swing.JComboBox; import javax.swing.JTextField; import javax.swing.SwingUtilities; import javax.swing.event.DocumentEvent; import javax.swing.event.DocumentListener; import javax.swing.table.DefaultTableModel;

/** Shared database engine supporting automatic payment generation, dynamic pricing, and appointment sync. */ public class Db { public static final String SEP = "\u001F"; public static final Map<String, String[]> META = new LinkedHashMap<>();

static {
    META.put("Pet Information", new String[]{"pets", "pet_id", "pet_name", "species", "breed", "age", "gender", "owner_name", "contact_number", "health_status", "vaccination_status"});
    META.put("Appointments", new String[]{"appointments", "appointment_id", "pet_name", "owner_name", "appointment_date", "appointment_time", "service", "veterinarian", "status"});
    META.put("Vaccinations", new String[]{"vaccinations", "vaccination_id", "pet_name", "vaccine_name", "date_given", "next_due_date", "dosage", "veterinarian", "status"});
    META.put("Payments", new String[]{"payments", "payment_id", "pet_name", "owner_name", "service", "amount", "payment_method", "payment_date", "status"});
}

public static String nextId(Connection c, String module, String prefix) throws SQLException {
    String[] m = META.get(module);
    int from = prefix.length() + 1;
    int max = 0;
    try (Statement s = c.createStatement();
         ResultSet rs = s.executeQuery("SELECT IFNULL(MAX(CAST(SUBSTRING(" + m[1] + "," + from + ") AS UNSIGNED)),0) FROM " + m[0])) {
        if (rs.next()) max = rs.getInt(1);
    }
    try (PreparedStatement ps = c.prepareStatement("SELECT IFNULL(MAX(CAST(SUBSTRING(record_id," + from + ") AS UNSIGNED)),0) FROM archive WHERE module_name=?")) {
        ps.setString(1, module);
        try (ResultSet rs = ps.executeQuery()) {
            if (rs.next()) max = Math.max(max, rs.getInt(1));
        }
    }
    return String.format("%s%03d", prefix, max + 1);
}
```

```
public static void fill(DefaultTableModel model, String sql, int cols) throws SQLException {
    model.setRowCount(0);
    try (Connection c = DBConnection.getConnection();
         Statement s = c.createStatement();
         ResultSet rs = s.executeQuery(sql)) {
        while (rs.next()) {
            Object[] row = new Object[cols];
            for (int i = 0; i < cols; i++) {
                String v = rs.getString(i + 1);
                row[i] = v == null ? "" : v;
            }
            model.addRow(row);
        }
    }
}


/** Exact clinic service pricing structure. */
public static double getServiceFee(String service) {
    if (service == null) return 500.00;
    switch (service.trim()) {
        case "Vaccination": return 800.00;
        case "Grooming": return 600.00;
        case "Surgery": return 2500.00;
        case "Dental Cleaning": return 1200.00;
        case "Deworming": return 400.00;
        case "Check-up":
        default: return 500.00;
    }
}


/** Scans all appointments and automatically creates or updates payments in the background. */
public static void syncAllAppointmentsToPayments() {
    try (Connection c = DBConnection.getConnection();
         Statement s = c.createStatement();
         ResultSet rs = s.executeQuery("SELECT pet_name, owner_name, service, appointment_date, status FROM appointments"))
        List<String[]> apts = new ArrayList<>();
        while (rs.next()) {
            apts.add(new String[]{
                rs.getString("pet_name"),
                rs.getString("owner_name"),
                rs.getString("service"),
                rs.getString("appointment_date"),
                rs.getString("status")
            });
        for (String[] apt : apts) {
            String pet = apt[0];
            String owner = apt[1];
            String service = apt[2];
            String date = apt[3];
            String status = apt[4
            double fee = getServiceFee(service);
            String targetPayStatus = "Completed".equalsIgnoreCase(status) ? "Paid" : "Pending";


            String checkSql = "SELECT payment_id, status FROM payments WHERE pet_name=? AND owner_name=? AND service=? AND payment_date=?";
            try (PreparedStatement ps = c.prepareStatement(checkSql)) {
                ps.setString(1, pet);
                ps.setString(2, owner);
                ps.setString(3, service);
                ps.setString(4, date);
                try (ResultSet prs = ps.executeQuery()) {
                    if (prs.next()) {
                        String payId = prs.getString(1);
                        String curPayStatus = prs.getString(2);
                        if ("Completed".equalsIgnoreCase(status) && !"Paid".equals(curPayStatus)) {
                            try (PreparedStatement up = c.prepareStatement("UPDATE payments SET status='Paid' WHERE payment_id=?")) {
                                up.setString(1, payId);
                                up.executeUpdate();
                            }
                        }
                    } else {
                        String newPayId = nextId(c, "Payments", "PAY-");
                        try (PreparedStatement ins = c.prepareStatement(
                                "INSERT INTO payments (payment_id, pet_name, owner_name, service, amount, payment_method, payment_date, status) VALUES (?,?,?,?,?,?,?,?)")) {
                            ins.setString(1, newPayId);
                            ins.setString(2, pet);
                            ins.setString(3, owner);
                            ins.setString(4, service);
                            ins.setBigDecimal(5, new BigDecimal(fee).setScale(2, RoundingMode.HALF_UP));
                            ins.setString(6, "Cash");
```

```
                        ins.setString(7, date);
                        ins.setString(8, targetPayStatus);
                        ins.executeUpdate();
                    }
                }
            }
        }
    }
    } catch (Exception e) {
        e.printStackTrace();
    }
}

public static Map<String, String[]> petDetails() throws SQLException {
    Map<String, String[]> map = new LinkedHashMap<>();
    String sql = Session.isAdmin()
            ? "SELECT pet_id, pet_name, owner_name FROM pets ORDER BY pet_id"
            : "SELECT pet_id, pet_name, owner_name FROM pets WHERE owner_name=? ORDER BY pet_id";

    try (Connection c = DBConnection.getConnection();
         PreparedStatement ps = c.prepareStatement(sql)) {
        if (!Session.isAdmin()) {
            ps.setString(1, Session.getFullName());
        }
        try (ResultSet rs = ps.executeQuery()) {
            while (rs.next()) {
                String id = rs.getString(1);
                String name = rs.getString(2);
                String owner = rs.getString(3) == null ? "" : rs.getString(3);
                map.put(id + " - " + name, new String[]{id, name, owner});
            }
        }
    }
    return map;
}

public static void bindPets(JComboBox<String> cb, JTextField owner) {
    Map<String, String[]> pets;
    try { pets = petDetails(); } catch (SQLException e) { pets = new LinkedHashMap<>(); }
    final Map<String, String[]> map = pets;
    final List<String> masterList = new ArrayList<>(map.keySet());

    cb.setEditable(true);
    cb.removeAllItems();
    for (ActionListener al : cb.getActionListeners()) cb.removeActionListener(al);

    cb.addItem("");
    for (String item : masterList) cb.addItem(item);

    JTextField editor = (JTextField) cb.getEditor().getEditorComponent();

    cb.addActionListener(e -> {
        Object selected = cb.getSelectedItem();
        if (owner != null) {
            if (selected != null && map.containsKey(selected.toString())) {
                owner.setText(map.get(selected.toString())[2]);
            } else if (!Session.isAdmin()) {
                owner.setText(Session.getFullName());
            } else {
                owner.setText("");
            }
        }
    });

    cb.setSelectedIndex(0);
    editor.setText("");
    if (owner != null) {
        owner.setText(Session.isAdmin() ? "" : Session.getFullName());
    }

    editor.getDocument().addDocumentListener(new DocumentListener() {
        private boolean updating = false;

        private void filter() {
            if (updating) return;
```

```
            SwingUtilities.invokeLater(() -> {
                updating = true;
                String text = editor.getText();
                int caret = editor.getCaretPosition();

                List<String> filtered = new ArrayList<>();
                if (text.trim().isEmpty()) {
                    filtered.add("");
                    filtered.addAll(masterList);
                } else {
                    for (String item : masterList) {
                        if (item.toLowerCase().contains(text.toLowerCase())) {
                            filtered.add(item);
                        }
                    }
                }

                cb.hidePopup();
                cb.removeAllItems();
                for (String f : filtered) cb.addItem(f);

                editor.setText(text);
                try { editor.setCaretPosition(Math.min(caret, text.length())); } catch (Exception ignored) {}

                if (!filtered.isEmpty() && !text.isEmpty() && editor.isFocusOwner()) {
                    cb.showPopup();
                }
                updating = false;
            });
        }

        public void insertUpdate(DocumentEvent e) { filter(); }
        public void removeUpdate(DocumentEvent e) { filter(); }
        public void changedUpdate(DocumentEvent e) { filter(); }
    });
}

public static void resetPetCombo(JComboBox<String> cb) {
    Map<String, String[]> pets;
    try { pets = petDetails(); } catch (SQLException e) { pets = new LinkedHashMap<>(); }
    cb.removeAllItems();
    cb.addItem("");
    for (String key : pets.keySet()) cb.addItem(key);
    cb.setSelectedIndex(0);
    JTextField editor = (JTextField) cb.getEditor().getEditorComponent();
    editor.setText("");
}

public static void selectPet(JComboBox<String> cb, String nameOrId) {
    if (nameOrId == null || nameOrId.trim().isEmpty()) return;
    String search = nameOrId.trim().toLowerCase();

    for (int i = 0; i < cb.getItemCount(); i++) {
        String item = cb.getItemAt(i);
        if (item == null) continue;
        String itemLower = item.toLowerCase();

        if (itemLower.equalsIgnoreCase(search) ||
            itemLower.endsWith(" - " + search) ||
            itemLower.startsWith(search + " - ")) {
            cb.setSelectedIndex(i);
            return;
        }
    }
}

public static void archiveRecord(String module, String id, String summary) throws SQLException {
    String[] m = META.get(module);
    String cols = String.join(",", Arrays.copyOfRange(m, 1, m.length));
    try (Connection c = DBConnection.getConnection()) {
        c.setAutoCommit(false);
        try {
            StringBuilder sb = new StringBuilder();
            try (PreparedStatement ps = c.prepareStatement("SELECT " + cols + " FROM " + m[0] + " WHERE " + m[1] + "=?")) {
                ps.setString(1, id);
```

```
            try (ResultSet rs = ps.executeQuery()) {
                if (!rs.next()) throw new SQLException("Record not found.");
                int n = rs.getMetaData().getColumnCount();
                for (int i = 1; i <= n; i++) {
                    if (i > 1) sb.append(SEP);
                    String v = rs.getString(i);
                    sb.append(v == null ? "" : v);
                }
            }
        }
        try (PreparedStatement ps = c.prepareStatement("INSERT INTO archive (module_name, record_id, summary, full_data) VALUES (?,?,?,?)")) {
            ps.setString(1, module);
            ps.setString(2, id);
            ps.setString(3, summary);
            ps.setString(4, sb.toString());
            ps.executeUpdate();
        }
        try (PreparedStatement ps = c.prepareStatement("DELETE FROM " + m[0] + " WHERE " + m[1] + "=?")) {
            ps.setString(1, id);
            ps.executeUpdate();
        }
        c.commit();
    } catch (SQLException e) {
        c.rollback();
        throw e;
    }
  }
}

public static void restoreRecord(int archiveId, String module, String fullData) throws SQLException {
    String[] m = META.get(module);
    if (m == null) throw new SQLException("Unknown module: " + module);
    String[] cols = Arrays.copyOfRange(m, 1, m.length);
    String[] v = fullData.contains(SEP) ? fullData.split(SEP, -1) : fullData.split(",", -1);
    if (v.length != cols.length) throw new SQLException("Archived data is incomplete and cannot be restored.");
    StringBuilder q = new StringBuilder();
    for (int i = 0; i < cols.length; i++) q.append(i == 0 ? "?" : ",?");
    try (Connection c = DBConnection.getConnection()) {
        c.setAutoCommit(false);
        try {
            try (PreparedStatement ps = c.prepareStatement("INSERT INTO " + m[0] + " (" + String.join(",", cols) + ") VALUES (" + q + ")")) {
                for (int i = 0; i < v.length; i++) ps.setString(i + 1, v[i]);
                ps.executeUpdate();
            }
            try (PreparedStatement ps = c.prepareStatement("DELETE FROM archive WHERE archive_id=?")) {
                ps.setInt(1, archiveId);
                ps.executeUpdate();
            }
            c.commit();
        } catch (SQLException e) {
            c.rollback();
            throw e;
        }
    }
}
 }
```

Export.java ×

```
package petpulse;

import java.io.; import java.nio.charset.StandardCharsets; import javax.swing.;

/** Exports what is shown in a JTable to a CSV file (opens in Excel). */ public class Export { public static void csv(JFrame parent, JTable table, String defaultName) { JFileChooser fc = new JFileChooser(); fc.setSelectedFile(new File(defaultName)); if (fc.showSaveDialog(parent) != JFileChooser.APPROVE_OPTION) return; File f = fc.getSelectedFile(); if (!f.getName().toLowerCase().endsWith(".csv")) f = new File(f.getParentFile(), f.getName() + ".csv"); try (Writer w = new OutputStreamWriter(new FileOutputStream(f), StandardCharsets.UTF_8)) { w.write('\uFEFF'); int cols = table.getColumnCount(); for (int c = 0; c < cols; c++) w.write(q(table.getColumnName(c)) + (c < cols - 1 ? "," : "\n")); for (int r = 0; r < table.getRowCount(); r++) for (int c = 0; c < cols; c++) w.write(q(String.valueOf(table.getValueAt(r, c))) + (c < cols - 1 ? "," : "\n")); UITheme.toast(parent, "Exported " + table.getRowCount() + " rows to " + f.getName(), true); } catch (IOException ex) { UITheme.showError(parent, ex); } }

private static String q(String s) {
    return "\"" + s.replace("\"", "\"\"") + "\"";
}


}
```

LoginFrame.java

```
package petpulse;

import java.awt.*; import java.sql.*; import javax.swing.*; import javax.swing.border.EmptyBorder;

/** Login screen (Admin / Client). */ public class LoginFrame extends JFrame { private final JTextField txtUser = UITheme.field(""); private final JPasswordField txtPass = UITheme.passwordField(); private final JLabel lblError = UITheme.label(" ", UITheme.FONT_SMALL, UITheme.ARCHIVE_RED); private final JButton btnLogin = UITheme.createBtn("Sign In", UITheme.DEEP_TEAL); private final JCheckBox chkShow = new JCheckBox("Show password");

public LoginFrame() {
    super("PetPulse - Login");
    setIconImage(Logo.image(64));
    setSize(980, 620);
    setResizable(false);
    setLocationRelativeTo(null);
    setDefaultCloseOperation(EXIT_ON_CLOSE);

    JPanel root = new JPanel(new BorderLayout());
    setContentPane(root);
    root.add(brandPanel(), BorderLayout.WEST);
    root.add(formPanel(), BorderLayout.CENTER);

    getRootPane().setDefaultButton(btnLogin);
    btnLogin.addActionListener(e -> doLogin());
    final char defEcho = txtPass.getEchoChar();
    chkShow.addActionListener(e -> txtPass.setEchoChar(chkShow.isSelected() ? (char) 0 : defEcho));
    SwingUtilities.invokeLater(() -> txtUser.requestFocusInWindow());
}

private JPanel brandPanel() {
    JPanel p = new JPanel(new GridBagLayout()) {
        @Override protected void paintComponent(Graphics g) {
            Graphics2D g2 = (Graphics2D) g.create();
            UITheme.smooth(g2);
            g2.setPaint(new GradientPaint(0, 0, UITheme.DARK_SLATE, 0, getHeight(), UITheme.blend(UITheme.DARK_SLATE, UITheme.DEEP_TEAL, 0.35f)));
            g2.fillRect(0, 0, getWidth(), getHeight());
            g2.setColor(new Color(13, 148, 136, 38));
            g2.fillOval(-120, getHeight() - 220, 380, 380);
            g2.setColor(new Color(20, 184, 166, 28));
            g2.fillOval(getWidth() - 160, -110, 300, 300);
            g2.dispose();
        }
    };
    p.setPreferredSize(new Dimension(430, 620));
    JPanel in = new JPanel();
    in.setOpaque(false);
    in.setLayout(new BoxLayout(in, BoxLayout.Y_AXIS));
    Logo logo = new Logo(92);
    logo.setAlignmentX(Component.LEFT_ALIGNMENT);
    in.add(logo);
    in.add(Box.createVerticalStrut(22));
    JLabel name = UITheme.label("PetPulse", new Font("Segoe UI", Font.BOLD, 38), Color.WHITE);
    name.setAlignmentX(Component.LEFT_ALIGNMENT);
    in.add(name);
    JLabel sub = UITheme.label("Pet Care and Information System", new Font("Segoe UI", Font.PLAIN, 15), UITheme.MUTED);
    sub.setAlignmentX(Component.LEFT_ALIGNMENT);
    in.add(sub);
    in.add(Box.createVerticalStrut(34));
    String[] points = {"Pet profiles and health records", "Appointments and vaccination tracking", "Billing, audit trail and safe archive"};
    for (String s : points) {
        JLabel l = UITheme.label("<html><span style='color:#14B8A6'>&#9679;</span>  " + s + "</html>", new Font("Segoe UI", Font.PLAIN, 14), Color.decode("#CBD5E1"));
        l.setBorder(new EmptyBorder(5, 0, 5, 0));
        l.setAlignmentX(Component.LEFT_ALIGNMENT);
        in.add(l);
    }
    p.add(in);
    return p;
}

private JPanel formPanel() {
    JPanel p = new JPanel(new GridBagLayout());
    p.setBackground(UITheme.COOL_OFF_WHITE);

    UITheme.RoundPanel card = new UITheme.RoundPanel(22, Color.WHITE);
    card.setLayout(new BoxLayout(card, BoxLayout.Y_AXIS));
    card.setBorder(new EmptyBorder(34, 36, 30, 36));
```

```
        card.setPreferredSize(new Dimension(400, 470));

        JLabel hi = UITheme.label("Welcome back", new Font("Segoe UI", Font.BOLD, 26), UITheme.NIGHT_NAVY);
        JLabel sub = UITheme.label("Sign in to continue to your dashboard", UITheme.FONT_REG, UITheme.STEEL_BLUE);
        hi.setAlignmentX(Component.LEFT_ALIGNMENT);
        sub.setAlignmentX(Component.LEFT_ALIGNMENT);
        card.add(hi);
        card.add(sub);
        card.add(Box.createVerticalStrut(24));

        JPanel u = UITheme.box("Username", txtUser);
        JPanel pw = UITheme.box("Password", txtPass);
        u.setAlignmentX(Component.LEFT_ALIGNMENT);
        pw.setAlignmentX(Component.LEFT_ALIGNMENT);
        card.add(u);
        card.add(Box.createVerticalStrut(14));
        card.add(pw);
        card.add(Box.createVerticalStrut(8));

        chkShow.setOpaque(false);
        chkShow.setFont(UITheme.FONT_REG);
        chkShow.setForeground(UITheme.STEEL_BLUE);
        chkShow.setFocusPainted(false);
        chkShow.setAlignmentX(Component.LEFT_ALIGNMENT);
        card.add(chkShow);
        card.add(Box.createVerticalStrut(6));
        lblError.setAlignmentX(Component.LEFT_ALIGNMENT);
        card.add(lblError);
        card.add(Box.createVerticalStrut(10));

        btnLogin.setPreferredSize(new Dimension(100, 44));
        JPanel bw = new JPanel(new BorderLayout());
        bw.setOpaque(false);
        bw.add(btnLogin);
        bw.setMaximumSize(new Dimension(Integer.MAX_VALUE, 44));
        bw.setAlignmentX(Component.LEFT_ALIGNMENT);
        card.add(bw);
        card.add(Box.createVerticalStrut(18));

        // Demo accounts hint (delete these 3 lines for the final release if you want)
        JLabel hint = UITheme.label("<html><div style='text-align:center'>Demo accounts<br>admin / admin123  |  client / client123</div></html>", new Font("Segoe UI", Font.PLAIN, 12),
    UITheme.MUTED);
        hint.setAlignmentX(Component.LEFT_ALIGNMENT);
        card.add(hint);

        p.add(card);
        return p;
    }

    private void fail(String msg) {
        lblError.setText(msg);
        final Point origin = getLocation();
        final int[] off = {-10, 10, -8, 8, -5, 5, -2, 2, 0};
        final int[] i = {0};
        Timer t = new Timer(30, null);
        t.addActionListener(e -> {
            if (i[0] >= off.length) { t.stop(); setLocation(origin); return; }
            setLocation(origin.x + off[i[0]++], origin.y);
        });
        t.start();
    }

    private void doLogin() {
        String u = txtUser.getText().trim();
        String p = new String(txtPass.getPassword());
        if (u.isEmpty() || p.isEmpty()) { fail("Please enter your username and password."); return; }
        lblError.setText(" ");
        btnLogin.setEnabled(false);
        try (Connection c = DBConnection.getConnection();
             PreparedStatement ps = c.prepareStatement("SELECT username, role, full_name FROM app_users WHERE username=? AND password_hash=?")) {
            ps.setString(1, u);
            ps.setString(2, Session.hash(p));
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    String user = rs.getString(1), role = rs.getString(2), name = rs.getString(3);
                    Session.login(user, name, role);
```

```
                DBConnection.log("System", "Login", "Logged in as " + role);
                new DashboardFrame().setVisible(true);
                dispose();
                return;
            }
        }
        fail("Incorrect username or password.");
    } catch (Exception ex) {
        ex.printStackTrace();
        fail("Cannot reach the database. Make sure MySQL is running.");
    } finally {
        btnLogin.setEnabled(true);
    }
}



}
```

Logo.java

```
package petpulse;

import java.awt.*; import java.awt.geom.*; import java.awt.image.BufferedImage; import javax.swing.JComponent;

/** PetPulse logo drawn with Java2D: a paw print whose pad carries a heartbeat (pulse) line. */ public class Logo extends JComponent { private final int size;

public Logo(int size) {
    this.size = size;
    setPreferredSize(new Dimension(size, size));
    setOpaque(false);
}

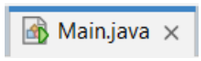
@Override protected void paintComponent(Graphics g) {
    Graphics2D g2 = (Graphics2D) g.create();
    UITheme.smooth(g2);
    draw(g2, 0, 0, size);
    g2.dispose();
}

public static void draw(Graphics2D g2, int x, int y, int s) {
    g2.translate(x, y);
    // badge
    g2.setPaint(new GradientPaint(0, 0, UITheme.BRIGHT_TEAL, s, s, UITheme.DEEP_TEAL));
    g2.fill(new RoundRectangle2D.Double(0, 0, s, s, s * 0.32, s * 0.32));
    // soft highlight
    g2.setColor(new Color(255, 255, 255, 38));
    g2.fill(new Ellipse2D.Double(-s * 0.2, -s * 0.45, s * 1.2, s * 0.9));
    // toes
    g2.setColor(Color.WHITE);
    g2.fill(new Ellipse2D.Double(s * 0.14, s * 0.34, s * 0.16, s * 0.21));
    g2.fill(new Ellipse2D.Double(s * 0.33, s * 0.17, s * 0.15, s * 0.23));
    g2.fill(new Ellipse2D.Double(s * 0.52, s * 0.17, s * 0.15, s * 0.23));
    g2.fill(new Ellipse2D.Double(s * 0.70, s * 0.34, s * 0.16, s * 0.21));
    // main pad
    g2.fill(new RoundRectangle2D.Double(s * 0.22, s * 0.47, s * 0.56, s * 0.38, s * 0.34, s * 0.34));
    // pulse line
    Path2D p = new Path2D.Double();
    p.moveTo(s * 0.27, s * 0.67);
    p.lineTo(s * 0.40, s * 0.67);
    p.lineTo(s * 0.45, s * 0.56);
    p.lineTo(s * 0.52, s * 0.79);
    p.lineTo(s * 0.57, s * 0.62);
    p.lineTo(s * 0.60, s * 0.67);
    p.lineTo(s * 0.73, s * 0.67);
    g2.setColor(UITheme.DEEP_TEAL);
    g2.setStroke(new BasicStroke(Math.max(1.5f, s * 0.045f), BasicStroke.CAP_ROUND, BasicStroke.JOIN_ROUND));
    g2.draw(p);
    g2.translate(-x, -y);
}

public static Image image(int s) {
    BufferedImage img = new BufferedImage(s, s, BufferedImage.TYPE_INT_ARGB);
    Graphics2D g = img.createGraphics();
    UITheme.smooth(g);
    draw(g, 0, 0, s);
    g.dispose();
    return img;
```

}

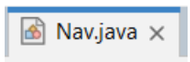
}



package petpulse;

import java.awt.EventQueue; import javax.swing.UIManager;

/** Program entry point. Run this file. */ public class Main { public static void main(String[] args) { System.setProperty("awt.useSystemAAFontSettings", "on"); System.setProperty("swing.aatext", "true"); EventQueue.invokeLater(() -> { try { UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName()); } catch (Exception ignored) { } UIManager.put("OptionPane.messageFont", UITheme.FONT_REG); UIManager.put("OptionPane.buttonFont", UITheme.FONT_BOLD); if (!DBConnection.init()) { System.exit(1); } new LoginFrame().setVisible(true); }); } }



package petpulse;

import java.awt.*; *import java.awt.event.MouseAdapter; import java.awt.event.MouseEvent; import java.awt.geom.Path2D; import java.awt.geom.RoundRectangle2D; import javax.swing.*;* import javax.swing.border.EmptyBorder;

/**

- Premium Modern IT Navigation Engine for PetPulse.
- Features built-in vector icon rendering, anti-aliased active pill indicators,
- responsive hover states, full-surface click target delegation, and seamless window transitions.

*/ public class Nav {

// Modern IT SaaS Color Palette (Tailwind Slate & Teal)
private static final Color BG_DARK = Color.decode("#0F172A"); // Slate 900
private static final Color BG_CARD = Color.decode("#1E293B"); // Slate 800
private static final Color TEAL_PRIMARY = Color.decode("#0D9488"); // Vibrant Teal 600
private static final Color TEAL_GLOW = Color.decode("#2DD4BF"); // Teal 400 Accent
private static final Color HOVER_BG = Color.decode("#1E293B"); // Slate 800 Hover
private static final Color TEXT_MUTED = Color.decode("#94A3B8"); // Muted Slate 400
private static final Color TEXT_LIGHT = Color.decode("#E2E8F0"); // Slate 200

public static JPanel createSidebar(JFrame currentFrame, String activeModule) {
boolean admin = Session.isAdmin();
JPanel sidebar = new JPanel(new BorderLayout());
sidebar.setBackground(BG_DARK);
sidebar.setPreferredSize(new Dimension(240, 760));

// --- TOP BRAND HEADER ---
JPanel brand = new JPanel(new FlowLayout(FlowLayout.LEFT, 14, 18));
brand.setOpaque(false);
brand.setBorder(new EmptyBorder(10, 10, 10, 10));
brand.add(new Logo(42));

JPanel brandText = new JPanel();
brandText.setOpaque(false);
brandText.setLayout(new BoxLayout(brandText, BoxLayout.Y_AXIS));
brandText.add(UITheme.label("PetPulse", new Font("Segoe UI", Font.BOLD, 22), Color.WHITE));
brandText.add(UITheme.label("Pet Care & Information", UITheme.FONT_SMALL, TEXT_MUTED));
brand.add(brandText);

// --- NAVIGATION MENU LINKS ---
JPanel menu = new JPanel();
menu.setOpaque(false);
menu.setLayout(new BoxLayout(menu, BoxLayout.Y_AXIS));
menu.setBorder(new EmptyBorder(10, 12, 10, 12));

JLabel lblCore = UITheme.label("CLINIC MENU", new Font("Segoe UI", Font.BOLD, 10), TEXT_MUTED);
lblCore.setBorder(new EmptyBorder(8, 12, 8, 12));
menu.add(lblCore);

// Core Self-Service Modules
addNavItem(menu, currentFrame, "Dashboard", activeModule, 1);
addNavItem(menu, currentFrame, "Pet Information", activeModule, 2);
addNavItem(menu, currentFrame, "Appointments", activeModule, 3);
addNavItem(menu, currentFrame, "Vaccinations", activeModule, 4);
addNavItem(menu, currentFrame, "Payments", activeModule, 5);

```
        // Admin Management Modules
        if (admin) {
            menu.add(Box.createVerticalStrut(14));
            JLabel lblAdmin = UITheme.label("ADMINISTRATION", new Font("Segoe UI", Font.BOLD, 10), TEXT_MUTED);
            lblAdmin.setBorder(new EmptyBorder(8, 12, 8, 12));
            menu.add(lblAdmin);

            addNavItem(menu, currentFrame, "Transaction History", activeModule, 6);
            addNavItem(menu, currentFrame, "Pet Records", activeModule, 7);
            addNavItem(menu, currentFrame, "Archive", activeModule, 8);
        }

        // --- USER PROFILE & LOGOUT FOOTER ---
        UITheme.RoundPanel userCard = new UITheme.RoundPanel(14, BG_CARD);
        userCard.setLineColor(BG_CARD);
        userCard.setLayout(new BorderLayout(10, 0));
        userCard.setBorder(new EmptyBorder(10, 12, 10, 12));

        String initial = Session.getFullName().isEmpty() ? "A" : Session.getFullName().substring(0, 1).toUpperCase();
        JLabel avatar = new JLabel(initial, SwingConstants.CENTER);
        avatar.setFont(UITheme.FONT_BOLD);
        avatar.setForeground(Color.WHITE);
        avatar.setOpaque(true);
        avatar.setBackground(TEAL_PRIMARY);
        avatar.setPreferredSize(new Dimension(36, 36));

        JPanel userInfo = new JPanel();
        userInfo.setOpaque(false);
        userInfo.setLayout(new BoxLayout(userInfo, BoxLayout.Y_AXIS));
        userInfo.add(UITheme.label(Session.getFullName().isEmpty() ? "Administrator" : Session.getFullName(), UITheme.FONT_BOLD, Color.WHITE));
        userInfo.add(UITheme.label(Session.getRole().isEmpty() ? "Admin" : Session.getRole(), UITheme.FONT_SMALL, TEXT_MUTED));

        userCard.add(avatar, BorderLayout.WEST);
        userCard.add(userInfo, BorderLayout.CENTER);

        JButton btnLogout = UITheme.createBtn("Logout", UITheme.ARCHIVE_RED);
        btnLogout.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
        btnLogout.addActionListener(e -> {
            Session.logout();
            Point pos = currentFrame.getLocation();
            Dimension size = currentFrame.getSize();
            LoginFrame login = new LoginFrame();
            login.setLocation(pos);
            login.setSize(size);
            login.setVisible(true);
            currentFrame.dispose();
        });

        JPanel footer = new JPanel(new BorderLayout(0, 10));
        footer.setOpaque(false);
        footer.setBorder(new EmptyBorder(12, 12, 16, 12));
        footer.add(userCard, BorderLayout.NORTH);
        footer.add(btnLogout, BorderLayout.SOUTH);

        sidebar.add(brand, BorderLayout.NORTH);
        sidebar.add(menu, BorderLayout.CENTER);
        sidebar.add(footer, BorderLayout.SOUTH);

        return sidebar;
    }

    private static void addNavItem(JPanel container, JFrame currentFrame, String moduleName, String activeModule, int iconType) {
        boolean isActive = moduleName.equalsIgnoreCase(activeModule);

        NavItemPanel item = new NavItemPanel(isActive, iconType, moduleName);
        item.setMaximumSize(new Dimension(Integer.MAX_VALUE, 42));

        // Full-surface mouse listener for clicks & hover
        MouseAdapter ma = new MouseAdapter() {
            @Override public void mouseEntered(MouseEvent e) {
                if (!isActive) {
                    item.setHover(true);
                }
            }
```

```
        @Override public void mouseExited(MouseEvent e) {
            if (!isActive) {
                item.setHover(false);
            }
        }
        @Override public void mouseClicked(MouseEvent e) {
            openFrame(currentFrame, moduleName);
        }
    };

    item.addMouseListener(ma);
    for (Component c : item.getComponents()) {
        c.addMouseListener(ma);
    }

    container.add(item);
    container.add(Box.createVerticalStrut(4));
}

private static void openFrame(JFrame currentFrame, String module) {
    String targetClass = module.replace(" ", "") + "Frame";
    if (currentFrame.getClass().getSimpleName().equals(targetClass)) return;

    Point pos = currentFrame.getLocation();
    Dimension size = currentFrame.getSize();
    int state = currentFrame.getExtendedState();

    SwingUtilities.invokeLater(() -> {
        JFrame newFrame = null;
        switch (module) {
            case "Dashboard": newFrame = new DashboardFrame(); break;
            case "Pet Information": newFrame = new PetInformationFrame(); break;
            case "Appointments": newFrame = new AppointmentsFrame(); break;
            case "Vaccinations": newFrame = new VaccinationsFrame(); break;
            case "Payments": newFrame = new PaymentsFrame(); break;
            case "Transaction History": newFrame = new TransactionHistoryFrame(); break;
            case "Pet Records": newFrame = new PetRecordsFrame(); break;
            case "Archive": newFrame = new ArchiveFrame(); break;
        }

        if (newFrame != null) {
            newFrame.setLocation(pos);
            newFrame.setSize(size);
            if ((state & Frame.MAXIMIZED_BOTH) == Frame.MAXIMIZED_BOTH) {
                newFrame.setExtendedState(Frame.MAXIMIZED_BOTH);
            }
            newFrame.setVisible(true);
            currentFrame.dispose();
        }
    });
}

/** Custom JPanel with vector icon painting, smooth rounded rects, and left glow bar. */
private static class NavItemPanel extends JPanel {
    private final boolean active;
    private boolean hover;
    private final int iconType;
    private final JLabel label;

    NavItemPanel(boolean active, int iconType, String text) {
        this.active = active;
        this.iconType = iconType;
        setOpaque(false);
        setLayout(new BorderLayout(12, 0));
        setBorder(new EmptyBorder(8, 16, 8, 14));
        setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));

        label = new JLabel(text);
        label.setFont(new Font("Segoe UI", active ? Font.BOLD : Font.PLAIN, 13));
        label.setForeground(active ? Color.WHITE : TEXT_LIGHT);
        label.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));

        // Component placeholder for icon area
        JComponent iconComp = new JComponent() {
            @Override protected void paintComponent(Graphics g) {
```

```
            super.paintComponent(g);
            Graphics2D g2 = (Graphics2D) g.create();
            g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);

            Color iconColor = active ? Color.WHITE : (hover ? Color.WHITE : TEXT_MUTED);
            g2.setColor(iconColor);
            drawVectorIcon(g2, iconType);
            g2.dispose();
        }
    };
    iconComp.setPreferredSize(new Dimension(20, 20));

    add(iconComp, BorderLayout.WEST);
    add(label, BorderLayout.CENTER);
}

void setHover(boolean h) {
    this.hover = h;
    label.setForeground(h ? Color.WHITE : TEXT_LIGHT);
    label.setFont(new Font("Segoe UI", (active || h) ? Font.BOLD : Font.PLAIN, 13));
    repaint();
}

@Override protected void paintComponent(Graphics g) {
    Graphics2D g2 = (Graphics2D) g.create();
    g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);

    int w = getWidth();
    int h = getHeight();

    if (active) {
        // Gradient Active Pill
        GradientPaint gp = new GradientPaint(0, 0, TEAL_PRIMARY, w, 0, Color.decode("#0F766E"));
        g2.setPaint(gp);
        g2.fill(new RoundRectangle2D.Float(0, 0, w, h, 12, 12));

        // Left Glow Indicator Line
        g2.setColor(TEAL_GLOW);
        g2.fill(new RoundRectangle2D.Float(2, 6, 4, h - 12, 4, 4));
    } else if (hover) {
        // Modern Slate Hover Pill
        g2.setColor(HOVER_BG);
        g2.fill(new RoundRectangle2D.Float(0, 0, w, h, 12, 12));
    }

    g2.dispose();
    super.paintComponent(g);
}

/** Vector Graphics Generator for Sidebar Icons. */
private void drawVectorIcon(Graphics2D g2, int type) {
    switch (type) {
        case 1: // Dashboard Grid
            g2.fillRoundRect(1, 1, 7, 7, 2, 2);
            g2.fillRoundRect(11, 1, 7, 7, 2, 2);
            g2.fillRoundRect(1, 11, 7, 7, 2, 2);
            g2.fillRoundRect(11, 11, 7, 7, 2, 2);
            break;
        case 2: // Pet Paw / Info
            g2.fillOval(7, 10, 7, 7);
            g2.fillOval(3, 6, 4, 4);
            g2.fillOval(8, 3, 4, 4);
            g2.fillOval(13, 6, 4, 4);
            break;
        case 3: // Appointments Calendar
            g2.setStroke(new BasicStroke(1.8f));
            g2.drawRoundRect(2, 3, 16, 15, 3, 3);
            g2.drawLine(2, 8, 18, 8);
            g2.fillOval(5, 11, 2, 2);
            g2.fillOval(9, 11, 2, 2);
            g2.fillOval(13, 11, 2, 2);
            g2.fillOval(5, 14, 2, 2);
            g2.fillOval(9, 14, 2, 2);
            break;
        case 4: // Vaccinations Shield
            Path2D shield = new Path2D.Float();
```

```
                shield.moveTo(10, 2);
                shield.lineTo(17, 5);
                shield.lineTo(17, 11);
                shield.curveTo(17, 16, 10, 18, 10, 18);
                shield.curveTo(10, 18, 3, 16, 3, 11);
                shield.lineTo(3, 5);
                shield.closePath();
                g2.setStroke(new BasicStroke(1.6f));
                g2.draw(shield);
                g2.fillRect(9, 7, 2, 6);
                g2.fillRect(7, 9, 6, 2);
                break;
            case 5: // Payments Credit Card
                g2.setStroke(new BasicStroke(1.8f));
                g2.drawRoundRect(1, 4, 18, 12, 3, 3);
                g2.fillRect(1, 8, 18, 3);
                g2.fillRect(4, 13, 4, 2);
                break;
            case 6: // Transaction History Clock
                g2.setStroke(new BasicStroke(1.8f));
                g2.drawOval(2, 2, 16, 16);
                g2.drawLine(10, 5, 10, 10);
                g2.drawLine(10, 10, 14, 10);
                break;
            case 7: // Pet Records File Folder
                g2.setStroke(new BasicStroke(1.8f));
                Path2D folder = new Path2D.Float();
                folder.moveTo(2, 4);
                folder.lineTo(8, 4);
                folder.lineTo(10, 6);
                folder.lineTo(18, 6);
                folder.lineTo(18, 16);
                folder.lineTo(2, 16);
                folder.closePath();
                g2.draw(folder);
                g2.drawLine(6, 10, 14, 10);
                g2.drawLine(6, 13, 11, 13);
                break;
            case 8: // Archive Storage Box
            default:
                g2.setStroke(new BasicStroke(1.8f));
                g2.drawRect(3, 7, 14, 11);
                g2.fillRect(1, 3, 18, 4);
                g2.drawLine(8, 12, 12, 12);
                break;
        }
    }
}


}
```

PaymentsFrame.java

```
package petpulse;

import java.awt.*; import java.math.BigDecimal; import java.math.RoundingMode; import java.sql.*; import java.time.LocalDate; import javax.swing.*; import javax.swing.table.DefaultTableModel;

/** Payments module with dynamic service pricing, pet booking auto-fill, and auto-synced totals. */ public class PaymentsFrame extends BaseFrame { private final JTextField txtId = UITheme.field("(auto)"), txtOwner = UITheme.field(""), txtAmount = UITheme.field("500.00"), txtDate = UITheme.field(LocalDate.now().toString()); private final JTextField txtSearch = UITheme.searchField("Search payments..."); private final JComboBox cbPet = UITheme.combo(); private final JComboBox cbService = UITheme.combo("Check-up", "Grooming", "Vaccination", "Surgery", "Dental Cleaning", "Deworming"); private final JComboBox cbMethod = UITheme.combo("Cash", "GCash", "Card"); private final JComboBox cbStatus = UITheme.combo("Paid", "Pending"); private final JLabel lblTotal = UITheme.label("Total Paid: PHP 0.00", UITheme.FONT_HEADER, UITheme.DEEP_TEAL); private final DefaultTableModel model = UITheme.model("Payment ID", "Pet Name", "Owner", "Service", "Amount (PHP)", "Method", "Date", "Status"); private final JTable table = new JTable(model);

public PaymentsFrame() {
    super("Payments", "Payments", "Record clinic billing and monitor what has been paid");
    boolean admin = Session.isAdmin();
    UITheme.lock(txtId);
    UITheme.lock(txtOwner);
    Db.bindPets(cbPet, txtOwner);


    // Real-time Service price auto-updater
    cbService.addActionListener(e -> {
        if (cbService.getSelectedItem() != null) {
            double fee = Db.getServiceFee(cbService.getSelectedItem().toString());
            txtAmount.setText(String.format("%.2f", fee));
        }
    });
```

```
// Pet Selection Listener: Fetches latest appointment service and auto-fills fields
cbPet.addActionListener(e -> {
    Object selectedPet = cbPet.getSelectedItem();
    if (selectedPet != null && !selectedPet.toString().trim().isEmpty()) {
        String petName = extractPetName(selectedPet);
        try (Connection c = DBConnection.getConnection();
             PreparedStatement ps = c.prepareStatement("SELECT service, appointment_date FROM appointments WHERE pet_name=? ORDER BY appointment_date DESC LIMIT 1")) {
            ps.setString(1, petName);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    String service = rs.getString(1);
                    cbService.setSelectedItem(service);
                    txtAmount.setText(String.format("%.2f", Db.getServiceFee(service)));
                    txtDate.setText(rs.getString(2));
                }
            }
        } catch (Exception ex) { }
    }
});

JPanel form = UITheme.formCard(4);
form.add(UITheme.box("Payment ID", txtId));
form.add(UITheme.box("Pet Name / ID", cbPet));
form.add(UITheme.box("Owner Name", txtOwner));
form.add(UITheme.box("Service", cbService));
form.add(UITheme.box("Amount (PHP)", txtAmount));
form.add(UITheme.box("Payment Method", cbMethod));
form.add(UITheme.box("Payment Date (YYYY-MM-DD)", txtDate));
form.add(UITheme.box("Status", cbStatus));

JButton btnAdd = UITheme.createBtn("Add", UITheme.BRIGHT_TEAL);
JButton btnUpd = UITheme.createBtn("Update", UITheme.STEEL_BLUE);
JButton btnArc = UITheme.createBtn("Archive", UITheme.ARCHIVE_RED);
JButton btnClr = UITheme.createBtn("Clear", UITheme.STEEL_BLUE);
JPanel btns = new JPanel(new FlowLayout(FlowLayout.LEFT, 10, 0));
btns.setOpaque(false);
if (admin) {
    btns.add(btnAdd);
    btns.add(btnUpd);
    btns.add(btnArc);
}
btns.add(btnClr);

JPanel east = new JPanel(new FlowLayout(FlowLayout.RIGHT, 18, 0));
east.setOpaque(false);
east.add(lblTotal);
east.add(txtSearch);

JPanel bar = new JPanel(new BorderLayout());
bar.setOpaque(false);
bar.add(btns, BorderLayout.WEST);
bar.add(east, BorderLayout.EAST);

JPanel top = new JPanel(new BorderLayout(0, 12));
top.setOpaque(false);
top.add(form, BorderLayout.NORTH);
top.add(bar, BorderLayout.SOUTH);

JPanel root = new JPanel(new BorderLayout(0, 14));
root.setOpaque(false);
root.add(top, BorderLayout.NORTH);
root.add(UITheme.tableCard(table), BorderLayout.CENTER);
setBody(root);

UITheme.attachSearch(txtSearch, table);
if (!admin) UITheme.readOnly(txtAmount, txtDate, cbPet, cbService, cbMethod, cbStatus);

table.getSelectionModel().addListSelectionListener(e -> {
    if (e.getValueIsAdjusting()) return;
    int r = table.getSelectedRow();
    if (r < 0 || r >= table.getRowCount()) return;
    try {
        r = table.convertRowIndexToModel(r);
        txtId.setText(v(r, 0));
        Db.selectPet(cbPet, v(r, 1));
```

```
            txtOwner.setText(v(r, 2));
            cbService.setSelectedItem(v(r, 3));
            txtAmount.setText(v(r, 4));
            cbMethod.setSelectedItem(v(r, 5));
            txtDate.setText(v(r, 6));
            cbStatus.setSelectedItem(v(r, 7));
        } catch (Exception ex) { }
    });

    btnClr.addActionListener(e -> clearForm());
    btnAdd.addActionListener(e -> addPayment());
    btnUpd.addActionListener(e -> updatePayment());
    btnArc.addActionListener(e -> archivePayment());

    loadData();
    clearForm();
}

private String v(int r, int c) {
    Object val = model.getValueAt(r, c);
    return val == null ? "" : String.valueOf(val);
}

private void clearForm() {
    txtId.setText("(auto)");
    txtAmount.setText("500.00");
    txtDate.setText(LocalDate.now().toString());
    cbService.setSelectedIndex(0);
    cbMethod.setSelectedIndex(0);
    cbStatus.setSelectedIndex(0);

    Db.resetPetCombo(cbPet);
    if (!Session.isAdmin()) {
        txtOwner.setText(Session.getFullName());
    } else {
        txtOwner.setText("");
    }
    table.clearSelection();
}

private String extractPetName(Object item) {
    if (item == null) return "";
    String s = item.toString().trim();
    return s.contains(" - ") ? s.split(" - ", 2)[1].trim() : s;
}

private String check() {
    Object selectedPet = cbPet.getSelectedItem();
    if (selectedPet == null || selectedPet.toString().trim().isEmpty()) {
        return "Please select or type a registered Pet Name / ID first.";
    }
    return Validation.firstError(Validation.amount(txtAmount.getText()), Validation.date(txtDate.getText(), "Payment date"));
}

private BigDecimal amount() {
    return new BigDecimal(txtAmount.getText().trim()).setScale(2, RoundingMode.HALF_UP);
}

private void addPayment() {
    String err = check();
    if (err != null) { UITheme.toast(this, err, false); return; }

    String petName = extractPetName(cbPet.getSelectedItem());
    try (Connection c = DBConnection.getConnection()) {
        String id = Db.nextId(c, "Payments", "PAY-");
        try (PreparedStatement ps = c.prepareStatement(
                "INSERT INTO payments (payment_id, pet_name, owner_name, service, amount, payment_method, payment_date, status) VALUES (?,?,?,?,?,?,?,?)")) {
            ps.setString(1, id);
            ps.setString(2, petName);
            ps.setString(3, txtOwner.getText());
            ps.setString(4, cbService.getSelectedItem().toString());
            ps.setBigDecimal(5, amount());
            ps.setString(6, cbMethod.getSelectedItem().toString());
            ps.setString(7, txtDate.getText().trim());
            ps.setString(8, cbStatus.getSelectedItem().toString());
```

```
            ps.executeUpdate();
        }
        DBConnection.log("Payments", "Add", id + " - " + petName + " (PHP " + amount() + ")");
        UITheme.toast(this, "Payment saved: " + id, true);
        clearForm();
        loadData();
    } catch (Exception ex) { UITheme.showError(this, ex); }
}


private void updatePayment() {
    if ("(auto)".equals(txtId.getText())) { UITheme.toast(this, "Select a payment from the table first.", false); return; }
    String err = check();
    if (err != null) { UITheme.toast(this, err, false); return; }


    String petName = extractPetName(cbPet.getSelectedItem());
    try (Connection c = DBConnection.getConnection();
         PreparedStatement ps = c.prepareStatement(
                 "UPDATE payments SET pet_name=?, owner_name=?, service=?, amount=?, payment_method=?, payment_date=?, status=? WHERE payment_id=?")) {
        ps.setString(1, petName);
        ps.setString(2, txtOwner.getText());
        ps.setString(3, cbService.getSelectedItem().toString());
        ps.setBigDecimal(4, amount());
        ps.setString(5, cbMethod.getSelectedItem().toString());
        ps.setString(6, txtDate.getText().trim());
        ps.setString(7, cbStatus.getSelectedItem().toString());
        ps.setString(8, txtId.getText());
        ps.executeUpdate();

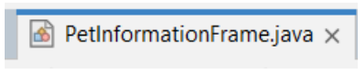

        DBConnection.log("Payments", "Update", txtId.getText() + " updated");
        UITheme.toast(this, "Payment updated.", true);
        clearForm();
        loadData();
    } catch (Exception ex) { UITheme.showError(this, ex); }
}


private void archivePayment() {
    if ("(auto)".equals(txtId.getText())) { UITheme.toast(this, "Select a payment from the table first.", false); return; }
    if (JOptionPane.showConfirmDialog(this, "Move " + txtId.getText() + " to the archive?", "Archive Payment", JOptionPane.YES_NO_OPTION) != JOptionPane.YES_OPTION) return;
    try {
        Db.archiveRecord("Payments", txtId.getText(), extractPetName(cbPet.getSelectedItem()) + " (PHP " + txtAmount.getText() + ")");
        DBConnection.log("Payments", "Archive", txtId.getText() + " archived");
        UITheme.toast(this, "Payment moved to archive.", true);
        clearForm();
        loadData();
    } catch (Exception ex) { UITheme.showError(this, ex); }
}


private void loadData() {
    try {
        Db.syncAllAppointmentsToPayments(); // Auto-imports all appointment invoices
        if (Session.isAdmin()) {
            Db.fill(model, "SELECT payment_id, pet_name, owner_name, service, amount, payment_method, payment_date, status FROM payments ORDER BY payment_id DESC", 8);
            try (Connection c = DBConnection.getConnection(); Statement s = c.createStatement();
                 ResultSet rs = s.executeQuery("SELECT IFNULL(SUM(amount),0) FROM payments WHERE status='Paid'")) {
                if (rs.next()) lblTotal.setText(String.format("Total Paid: PHP %,.2f", rs.getDouble(1)));
            }
        } else {
            String owner = Session.getFullName();
            model.setRowCount(0);
            try (Connection c = DBConnection.getConnection();
                 PreparedStatement ps = c.prepareStatement(
                   "SELECT payment_id, pet_name, owner_name, service, amount, payment_method, payment_date, status FROM payments WHERE owner_name=? ORDER BY payment_id DESC")) {
                ps.setString(1, owner);
                try (ResultSet rs = ps.executeQuery()) {
                    while (rs.next()) {
                        Object[] row = new Object[8];
                        for (int i = 0; i < 8; i++) {
                            String val = rs.getString(i + 1);
                            row[i] = val == null ? "" : val;
                        }
                        model.addRow(row);
                    }
                }
            }
            try (Connection c = DBConnection.getConnection();
                 PreparedStatement ps = c.prepareStatement("SELECT IFNULL(SUM(amount),0) FROM payments WHERE owner_name=? AND status='Paid'")) {
                ps.setString(1, owner);
                try (ResultSet rs = ps.executeQuery()) {
```

```
                if (rs.next()) lblTotal.setText(String.format("Total Paid: PHP %,.2f", rs.getDouble(1)));
              }
            }
          }
      } catch (Exception e) {
          e.printStackTrace();
          UITheme.toast(this, "Could not load payments. Is MySQL running?", false);
      }
  }


}
```




```
package petpulse;

import java.awt.*; import java.sql.*; import javax.swing.*; import javax.swing.table.DefaultTableModel;

/** Pet Information & Profile Registry Module. */ public class PetInformationFrame extends BaseFrame { private final JTextField txtId = UITheme.field("(auto)"); private final JTextField txtName = UITheme.field(""), txtBreed = UITheme.field(""), txtAge = UITheme.field(""), txtOwner = UITheme.field(""), txtContact = UITheme.field(""); private final JTextField txtSearch = UITheme.searchField("Search pets..."); private final JComboBox cbSpecies = UITheme.combo("Dog", "Cat", "Bird", "Rabbit", "Hamster", "Other"); private final JComboBox cbGender = UITheme.combo("Male", "Female"); private final JComboBox cbHealth = UITheme.combo("Healthy", "Under Treatment", "Critical"); private final JComboBox cbVacc = UITheme.combo("Vaccinated", "Partially Vaccinated", "Not Vaccinated"); private final DefaultTableModel model = UITheme.model("Pet ID", "Pet Name", "Species", "Breed", "Age", "Gender", "Owner", "Contact", "Health", "Vaccination"); private final JTable table = new JTable(model);

public PetInformationFrame() {
    super("Pet Information", "Pet Information", "Register pets, update profiles and monitor health status");
    boolean admin = Session.isAdmin();
    UITheme.lock(txtId);

    if (!admin) {
        txtOwner.setText(Session.getFullName());
        UITheme.lock(txtOwner);
    }

    JPanel form = UITheme.formCard(4);
    form.add(UITheme.box("Pet ID", txtId));
    form.add(UITheme.box("Pet Name", txtName));
    form.add(UITheme.box("Species", cbSpecies));
    form.add(UITheme.box("Breed", txtBreed));
    form.add(UITheme.box("Age (years)", txtAge));
    form.add(UITheme.box("Gender", cbGender));
    form.add(UITheme.box("Health Status", cbHealth));
    form.add(UITheme.box("Vaccination Status", cbVacc));
    form.add(UITheme.box("Owner Name", txtOwner));
    form.add(UITheme.box("Contact Number", txtContact));

    JButton btnAdd = UITheme.createBtn(admin ? "Add Pet" : "Register Pet", UITheme.BRIGHT_TEAL);
    JButton btnUpd = UITheme.createBtn("Update", UITheme.STEEL_BLUE);
    JButton btnArc = UITheme.createBtn("Archive", UITheme.ARCHIVE_RED);
    JButton btnClr = UITheme.createBtn("Clear", UITheme.STEEL_BLUE);

    JPanel btns = new JPanel(new FlowLayout(FlowLayout.LEFT, 10, 0));
    btns.setOpaque(false);
    btns.add(btnAdd);
    btns.add(btnUpd);
    btns.add(btnArc);
    btns.add(btnClr);

    JPanel bar = new JPanel(new BorderLayout());
    bar.setOpaque(false);
    bar.add(btns, BorderLayout.WEST);
    bar.add(txtSearch, BorderLayout.EAST);

    JPanel top = new JPanel(new BorderLayout(0, 12));
    top.setOpaque(false);
    top.add(form, BorderLayout.NORTH);
    top.add(bar, BorderLayout.SOUTH);

    JPanel root = new JPanel(new BorderLayout(0, 14));
    root.setOpaque(false);
    root.add(top, BorderLayout.NORTH);
    root.add(UITheme.tableCard(table), BorderLayout.CENTER);
    setBody(root);
```

```
        UITheme.attachSearch(txtSearch, table);

        table.getSelectionModel().addListSelectionListener(e -> {
            if (e.getValueIsAdjusting()) return;
            int r = table.getSelectedRow();
            if (r < 0 || r >= table.getRowCount()) return;
            try {
                fillForm(table.convertRowIndexToModel(r));
            } catch (Exception ex) { }
        });

        btnClr.addActionListener(e -> clearForm());
        btnAdd.addActionListener(e -> addPet());
        btnUpd.addActionListener(e -> updatePet());
        btnArc.addActionListener(e -> archivePet());
        loadData();
    }

    private String v(int r, int c) {
        Object val = model.getValueAt(r, c);
        return val == null ? "" : String.valueOf(val);
    }

    private void fillForm(int r) {
        txtId.setText(v(r, 0));
        txtName.setText(v(r, 1));
        cbSpecies.setSelectedItem(v(r, 2));
        txtBreed.setText(v(r, 3));
        txtAge.setText(v(r, 4));
        cbGender.setSelectedItem(v(r, 5));
        txtOwner.setText(v(r, 6));
        txtContact.setText(v(r, 7));
        cbHealth.setSelectedItem(v(r, 8));
        cbVacc.setSelectedItem(v(r, 9));
    }

    private void clearForm() {
        txtId.setText("(auto)");
        txtName.setText("");
        txtBreed.setText("");
        txtAge.setText("");
        if (Session.isAdmin()) {
            txtOwner.setText("");
        } else {
            txtOwner.setText(Session.getFullName());
        }
        txtContact.setText("");
        cbSpecies.setSelectedIndex(0);
        cbGender.setSelectedIndex(0);
        cbHealth.setSelectedIndex(0);
        cbVacc.setSelectedIndex(0);
        table.clearSelection();
    }

    private String check() {
        return Validation.firstError(
            Validation.required(txtName.getText(), "Pet name"),
            Validation.required(txtBreed.getText(), "Breed"),
            Validation.age(txtAge.getText()),
            Validation.required(txtOwner.getText(), "Owner name"),
            Validation.phone(txtContact.getText()));
    }

    private void addPet() {
        String err = check();
        if (err != null) { UITheme.toast(this, err, false); return; }
        String ownerName = Session.isAdmin() ? txtOwner.getText().trim() : Session.getFullName();
        try (Connection c = DBConnection.getConnection()) {
            String id = Db.nextId(c, "Pet Information", "PET-");
            try (PreparedStatement ps = c.prepareStatement(
                    "INSERT INTO pets (pet_id, pet_name, species, breed, age, gender, owner_name, contact_number, health_status, vaccination_status) VALUES (?,?,?,?,?,?,?,?,?,?)")) {
                ps.setString(1, id);
                ps.setString(2, txtName.getText().trim());
                ps.setString(3, cbSpecies.getSelectedItem().toString());
                ps.setString(4, txtBreed.getText().trim());
                ps.setInt(5, Integer.parseInt(txtAge.getText().trim()));
                ps.setString(6, cbGender.getSelectedItem().toString());
```

```
            ps.setString(7, ownerName);
            ps.setString(8, txtContact.getText().trim());
            ps.setString(9, cbHealth.getSelectedItem().toString());
            ps.setString(10, cbVacc.getSelectedItem().toString());
            ps.executeUpdate();
        }
        DBConnection.log("Pet Information", "Add", id + " - " + txtName.getText().trim() + " (" + ownerName + ")");
        UITheme.toast(this, "Pet registered: " + id, true);
        clearForm();
        loadData();
    } catch (Exception ex) { UITheme.showError(this, ex); }
}

private void updatePet() {
    if ("(auto)".equals(txtId.getText())) { UITheme.toast(this, "Select a pet from the table first.", false); return; }
    String err = check();
    if (err != null) { UITheme.toast(this, err, false); return; }

    String ownerName = Session.isAdmin() ? txtOwner.getText().trim() : Session.getFullName();
    try (Connection c = DBConnection.getConnection();
         PreparedStatement ps = c.prepareStatement(
             "UPDATE pets SET pet_name=?, species=?, breed=?, age=?, gender=?, owner_name=?, contact_number=?, health_status=?, vaccination_status=? WHERE pet_id=?")) {
        ps.setString(1, txtName.getText().trim());
        ps.setString(2, cbSpecies.getSelectedItem().toString());
        ps.setString(3, txtBreed.getText().trim());
        ps.setInt(4, Integer.parseInt(txtAge.getText().trim()));
        ps.setString(5, cbGender.getSelectedItem().toString());
        ps.setString(6, ownerName);
        ps.setString(7, txtContact.getText().trim());
        ps.setString(8, cbHealth.getSelectedItem().toString());
        ps.setString(9, cbVacc.getSelectedItem().toString());
        ps.setString(10, txtId.getText());
        ps.executeUpdate();

        DBConnection.log("Pet Information", "Update", txtId.getText() + " updated");
        UITheme.toast(this, "Pet profile updated.", true);
        clearForm();
        loadData();
    } catch (Exception ex) { UITheme.showError(this, ex); }
}

private void archivePet() {
    if ("(auto)".equals(txtId.getText())) { UITheme.toast(this, "Select a pet from the table first.", false); return; }
    if (JOptionPane.showConfirmDialog(this, "Move " + txtName.getText() + " to the archive?", "Archive Pet", JOptionPane.YES_NO_OPTION) != JOptionPane.YES_OPTION) return;
    try {
        Db.archiveRecord("Pet Information", txtId.getText(), txtName.getText() + " (" + cbSpecies.getSelectedItem() + ")");
        DBConnection.log("Pet Information", "Archive", txtId.getText() + " archived");
        UITheme.toast(this, "Pet moved to archive.", true);
        clearForm();
        loadData();
    } catch (Exception ex) { UITheme.showError(this, ex); }
}

private void loadData() {
    try {
        if (Session.isAdmin()) {
            Db.fill(model, "SELECT pet_id, pet_name, species, breed, age, gender, owner_name, contact_number, health_status, vaccination_status FROM pets ORDER BY pet_id", 10);
        } else {
            String owner = Session.getFullName();
            model.setRowCount(0);
            try (Connection c = DBConnection.getConnection();
                 PreparedStatement ps = c.prepareStatement(
                   "SELECT pet_id, pet_name, species, breed, age, gender, owner_name, contact_number, health_status, vaccination_status FROM pets WHERE owner_name=? ORDER BY pet_id")) {
                ps.setString(1, owner);
                try (ResultSet rs = ps.executeQuery()) {
                    while (rs.next()) {
                        Object[] row = new Object[10];
                        for (int i = 0; i < 10; i++) {
                            String val = rs.getString(i + 1);
                            row[i] = val == null ? "" : val;
                        }
                        model.addRow(row);
                    }
                }
            }
        }
    } catch (Exception e) {
        e.printStackTrace();
```

```
            UITheme.toast(this, "Could not load pets. Is MySQL running?", false);
        }
    }
}


}
```

PetRecordsFrame.java

```
package petpulse;

import java.awt.*; import java.sql.*; import javax.swing.*; import javax.swing.border.EmptyBorder; import javax.swing.table.DefaultTableModel;

/** View-only master list of pets with a profile card (Admin only). */ public class PetRecordsFrame extends BaseFrame { private final DefaultTableModel model = UITheme.model("Pet ID", "Pet Name", "Species",
"Owner Name"); private final JTable table = new JTable(model); private final JTextField txtSearch = UITheme.searchField("Search records..."); private final JLabel lblCount = UITheme.label("0 pets",
UITheme.FONT_HEADER, UITheme.DEEP_TEAL); private final JLabel lblProfile = new JLabel();

public PetRecordsFrame() {
    super("Pet Records", "Pet Records", "Read-only master list for fast lookup without accidental editing");     JButton btnExport = UITheme.createBtn("Export CSV", UITheme.STEEL_BLUE);
    JPanel west = new JPanel(new FlowLayout(FlowLayout.LEFT, 14, 0));
    west.setOpaque(false);
    west.add(btnExport);
    west.add(lblCount);
    JPanel bar = new JPanel(new BorderLayout());
    bar.setOpaque(false);
    bar.add(west, BorderLayout.WEST);
    bar.add(txtSearch, BorderLayout.EAST);


    UITheme.RoundPanel profile = new UITheme.RoundPanel(18, UITheme.CARD_BG);
    profile.setLayout(new BorderLayout());
    profile.setBorder(new EmptyBorder(18, 20, 18, 20));
    profile.setPreferredSize(new Dimension(330, 100));
    lblProfile.setVerticalAlignment(SwingConstants.TOP);
    lblProfile.setFont(UITheme.FONT_REG);
    showProfile(null);
    profile.add(lblProfile, BorderLayout.CENTER);


    JPanel center = new JPanel(new BorderLayout(18, 0));
    center.setOpaque(false);
    center.add(UITheme.tableCard(table), BorderLayout.CENTER);
    center.add(profile, BorderLayout.EAST);


    JPanel root = new JPanel(new BorderLayout(0, 14));
    root.setOpaque(false);
    root.add(bar, BorderLayout.NORTH);
    root.add(center, BorderLayout.CENTER);
    setBody(root);


    UITheme.attachSearch(txtSearch, table);
    btnExport.addActionListener(e -> Export.csv(this, table, "pet_records.csv"));
    table.getSelectionModel().addListSelectionListener(e -> {
        if (e.getValueIsAdjusting()) return;
        int r = table.getSelectedRow();
        showProfile(r < 0 ? null : String.valueOf(model.getValueAt(table.convertRowIndexToModel(r), 0)));
    });
    loadData();
}

private static String esc(String s) {
    return s == null ? "" : s.replace("&", "&amp;").replace("<", "&lt;").replace(">", "&gt;");
}

private String row(String k, String v) {
    return "<tr><td style='color:#475569;padding:4px 10px 4px 0'>" + k + "</td><td style='color:#020617'><b>" + esc(v) + "</b></td></tr>";
}

private void showProfile(String id) {
    if (id == null) {
        lblProfile.setText("<html><div style='width:250px'><span style='font-size:15px'><b>Pet Profile</b></span><br><br>"
            + "<span style='color:#475569'>Select a pet from the list to see its full profile and activity.</span></div></html>");
        return;
    }
    try (Connection c = DBConnection.getConnection();
         PreparedStatement ps = c.prepareStatement("SELECT pet_id, pet_name, species, breed, age, gender, owner_name, contact_number, health_status, vaccination_status, "
            + "(SELECT COUNT(*) FROM appointments a WHERE a.pet_name = p.pet_name), "
            + "(SELECT COUNT(*) FROM vaccinations v WHERE v.pet_name = p.pet_name), "
```

```
            + "(SELECT COUNT(*) FROM payments y WHERE y.pet_name = p.pet_name) FROM pets p WHERE pet_id=?")) {
        ps.setString(1, id);
        try (ResultSet rs = ps.executeQuery()) {
            if (!rs.next()) { showProfile(null); return; }
            StringBuilder sb = new StringBuilder("<html><div style='width:260px'><span style='font-size:16px'><b>")
                .append(esc(rs.getString(2))).append("</b></span><br><span style='color:#0D9488'>").append(esc(rs.getString(1))).append("</span><br><br><table>");
            sb.append(row("Species", rs.getString(3))).append(row("Breed", rs.getString(4))).append(row("Age", rs.getString(5) + " yr"))
              .append(row("Gender", rs.getString(6))).append(row("Owner", rs.getString(7))).append(row("Contact", rs.getString(8)))
              .append(row("Health", rs.getString(9))).append(row("Vaccination", rs.getString(10)))
              .append("</table><br><span style='color:#475569'>Activity</span><table>")
              .append(row("Appointments", rs.getString(11))).append(row("Vaccinations", rs.getString(12))).append(row("Payments", rs.getString(13)))
              .append("</table></div></html>");
            lblProfile.setText(sb.toString());
        }
    } catch (Exception ex) {
        ex.printStackTrace();
        lblProfile.setText("<html>Could not load profile.</html>");
    }
}


private void loadData() {
    try {
        Db.fill(model, "SELECT pet_id, pet_name, species, owner_name FROM pets ORDER BY pet_id", 4);
        lblCount.setText(model.getRowCount() + (model.getRowCount() == 1 ? " pet" : " pets"));
    } catch (Exception e) {
        e.printStackTrace();
        UITheme.toast(this, "Could not load records. Is MySQL running?", false);
    }
}



}
```

Session.java ×

```
package petpulse;

import java.security.MessageDigest;

/** Keeps track of who is logged in (Admin or Client). */ public class Session { private static String username = ""; private static String fullName = ""; private static String role = "";

public static void login(String user, String name, String r) {
    username = user == null ? "" : user;
    fullName = name == null ? "" : name;
    role = r == null ? "" : r;
}


public static void logout() { username = ""; fullName = ""; role = ""; }
public static boolean isAdmin() { return "Admin".equalsIgnoreCase(role); }
public static String getUsername() { return username; }
public static String getFullName() { return fullName; }
public static String getRole() { return role; }


/** SHA-256 so passwords are never stored as plain text. */
public static String hash(String text) {
    try {
        MessageDigest md = MessageDigest.getInstance("SHA-256");
        byte[] d = md.digest(text.getBytes("UTF-8"));
        StringBuilder sb = new StringBuilder();
        for (byte b : d) sb.append(String.format("%02x", b));
        return sb.toString();
    } catch (Exception e) {
        throw new RuntimeException(e);
    }
}



}
```

TransactionHistoryFrame.java ×

```
package petpulse;

import java.awt.; import java.sql.; import javax.swing.*; import javax.swing.table.DefaultTableModel;
```

```
/** Audit trail of every action in the system (Admin only). */ public class TransactionHistoryFrame extends BaseFrame { private final DefaultTableModel model = UITheme.model("Date & Time", "Module", "Action",
"Details"); private final JTable table = new JTable(model); private final JTextField txtSearch = UITheme.searchField("Search history...");

public TransactionHistoryFrame() {
    super("Transaction History", "Transaction History", "Automatic audit log of every login, change and archive action");
    JButton btnRefresh = UITheme.createBtn("Refresh", UITheme.BRIGHT_TEAL);
    JButton btnExport = UITheme.createBtn("Export CSV", UITheme.STEEL_BLUE);
    JButton btnClear = UITheme.createBtn("Clear History", UITheme.ARCHIVE_RED);
    JPanel btns = new JPanel(new FlowLayout(FlowLayout.LEFT, 10, 0));
    btns.setOpaque(false);
    btns.add(btnRefresh);
    btns.add(btnExport);
    btns.add(btnClear);
    JPanel bar = new JPanel(new BorderLayout());
    bar.setOpaque(false);
    bar.add(btns, BorderLayout.WEST);
    bar.add(txtSearch, BorderLayout.EAST);

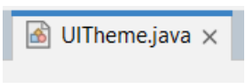
    JPanel root = new JPanel(new BorderLayout(0, 14));
    root.setOpaque(false);
    root.add(bar, BorderLayout.NORTH);
    root.add(UITheme.tableCard(table), BorderLayout.CENTER);
    setBody(root);

    UITheme.attachSearch(txtSearch, table);
    btnRefresh.addActionListener(e -> { loadLogs(); UITheme.toast(this, "History refreshed.", true); });
    btnExport.addActionListener(e -> Export.csv(this, table, "transaction_history.csv"));
    btnClear.addActionListener(e -> {
        if (JOptionPane.showConfirmDialog(this, "Permanently clear all history logs?", "Clear History", JOptionPane.YES_NO_OPTION) != JOptionPane.YES_OPTION) return;
        try (Connection c = DBConnection.getConnection(); Statement s = c.createStatement()) {
            s.executeUpdate("DELETE FROM transaction_history");
        } catch (Exception ex) { UITheme.showError(this, ex); return; }
        DBConnection.log("Transaction History", "Clear", "History cleared");
        loadLogs();
        UITheme.toast(this, "History cleared.", true);
    });
    loadLogs();
}

private void loadLogs() {
    try {
        Db.fill(model, "SELECT DATE_FORMAT(timestamp,'%Y-%m-%d %H:%i:%s'), module_name, action_type, details FROM transaction_history ORDER BY timestamp DESC, log_id DESC", 4);
    } catch (Exception e) {
        e.printStackTrace();
        UITheme.toast(this, "Could not load history. Is MySQL running?", false);
    }
}



}
```




```
package petpulse;

import java.awt.; import java.awt.event.; import java.awt.geom.RoundRectangle2D; import java.util.regex.Pattern; import javax.swing.; import javax.swing.border.AbstractBorder; import
javax.swing.border.EmptyBorder; import javax.swing.event.DocumentEvent; import javax.swing.event.DocumentListener; import javax.swing.plaf.basic.BasicComboBoxUI; import javax.swing.table.;

/** Slate & Modern Teal design system: colors, fonts and all the rounded / animated components. */ public class UITheme {

// ===== COLOR PALETTE (unchanged) =====
public static final Color DARK_SLATE = Color.decode("#0F172A");   // Sidebar BG
public static final Color DEEP_TEAL = Color.decode("#0D9488");    // Active Item
public static final Color COOL_OFF_WHITE = Color.decode("#F1F5F9"); // Main Screen BG
public static final Color BRIGHT_TEAL = Color.decode("#14B8A6");  // Add Button
public static final Color STEEL_BLUE = Color.decode("#475569");   // Secondary Buttons
public static final Color ARCHIVE_RED = Color.decode("#DC2626");  // Archive Button
public static final Color NIGHT_NAVY = Color.decode("#020617");   // Primary Text
public static final Color CARD_BG = Color.WHITE;
// helper tones
public static final Color SIDEBAR_HOVER = Color.decode("#1E293B");
public static final Color BORDER = Color.decode("#E2E8F0");
public static final Color BORDER_DARK = Color.decode("#CBD5E1");
public static final Color BLUE = Color.decode("#3B82F6");
public static final Color AMBER = Color.decode("#F59E0B");
public static final Color MUTED = Color.decode("#94A3B8");
```

```
    public static final Font FONT_TITLE = new Font("Segoe UI", Font.BOLD, 26);
    public static final Font FONT_HEADER = new Font("Segoe UI", Font.BOLD, 17);
    public static final Font FONT_BOLD = new Font("Segoe UI", Font.BOLD, 13);
    public static final Font FONT_REG = new Font("Segoe UI", Font.PLAIN, 13);
    public static final Font FONT_SMALL = new Font("Segoe UI", Font.BOLD, 12);

    public static void smooth(Graphics2D g) {
        g.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);
        g.setRenderingHint(RenderingHints.KEY_TEXT_ANTIALIASING, RenderingHints.VALUE_TEXT_ANTIALIAS_ON);
        g.setRenderingHint(RenderingHints.KEY_STROKE_CONTROL, RenderingHints.VALUE_STROKE_PURE);
    }

    public static Color blend(Color a, Color b, float t) {
        t = Math.max(0f, Math.min(1f, t));
        return new Color((int) (a.getRed() + (b.getRed() - a.getRed()) * t),
                         (int) (a.getGreen() + (b.getGreen() - a.getGreen()) * t),
                         (int) (a.getBlue() + (b.getBlue() - a.getBlue()) * t));
    }

    public static JLabel label(String text, Font f, Color c) {
        JLabel l = new JLabel(text);
        l.setFont(f);
        l.setForeground(c);
        return l;
    }

    // ===== BUTTONS =====
    public static JButton createBtn(String text, Color bg) {
        return new RoundButton(text, bg);
    }

    /** Rounded button with a smooth hover animation. */
    public static class RoundButton extends JButton {
        private final Color base;
        private float hover = 0f;
        private boolean inside = false;
        private final Timer timer;

        public RoundButton(String text, Color bg) {
            super(text);
            this.base = bg;
            setFont(FONT_BOLD);
            setForeground(Color.WHITE);
            setContentAreaFilled(false);
            setBorderPainted(false);
            setFocusPainted(false);
            setOpaque(false);
            setBorder(new EmptyBorder(9, 18, 9, 18));
            setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
            timer = new Timer(15, e -> {
                float target = inside ? 1f : 0f;
                hover += (target - hover) * 0.28f;
                if (Math.abs(target - hover) < 0.02f) { hover = target; ((Timer) e.getSource()).stop(); }
                repaint();
            });
            addMouseListener(new MouseAdapter() {
                @Override public void mouseEntered(MouseEvent e) { inside = true; timer.start(); }
                @Override public void mouseExited(MouseEvent e) { inside = false; timer.start(); }
            });
        }

        @Override protected void paintComponent(Graphics g) {
            Graphics2D g2 = (Graphics2D) g.create();
            smooth(g2);
            Color c = blend(base, Color.WHITE, hover * 0.16f);
            if (getModel().isPressed()) c = c.darker();
            if (!isEnabled()) c = blend(base, Color.WHITE, 0.6f);
            g2.setColor(c);
            g2.fillRoundRect(0, 0, getWidth(), getHeight(), 14, 14);
            g2.dispose();
            super.paintComponent(g);
        }
    }

    // ===== BORDERS / PANELS =====
    public static class RoundBorder extends AbstractBorder {
```

```
    public Color color;
    private final int arc, pv, pl, pr;

    public RoundBorder(Color color, int arc, int pv, int pl, int pr) {
        this.color = color; this.arc = arc; this.pv = pv; this.pl = pl; this.pr = pr;
    }

    @Override public void paintBorder(Component c, Graphics g, int x, int y, int w, int h) {
        Graphics2D g2 = (Graphics2D) g.create();
        smooth(g2);
        g2.setColor(color);
        g2.drawRoundRect(x, y, w - 1, h - 1, arc, arc);
        g2.dispose();
    }

    @Override public Insets getBorderInsets(Component c) { return new Insets(pv, pl, pv, pr); }
    @Override public Insets getBorderInsets(Component c, Insets i) { i.set(pv, pl, pv, pr); return i; }
}

/** White rounded card. Children are clipped to the rounded corners. */
public static class RoundPanel extends JPanel {
    private final int arc;
    private final Color fill;
    private Color line = BORDER;

    public RoundPanel(int arc, Color fill) {
        this.arc = arc;
        this.fill = fill;
        setOpaque(false);
    }

    public void setLineColor(Color c) { line = c; }

    @Override protected void paintComponent(Graphics g) {
        Graphics2D g2 = (Graphics2D) g.create();
        smooth(g2);
        g2.setColor(fill);
        g2.fillRoundRect(0, 0, getWidth() - 1, getHeight() - 1, arc, arc);
        g2.dispose();
    }

    @Override protected void paintChildren(Graphics g) {
        Graphics2D g2 = (Graphics2D) g.create();
        g2.setClip(new RoundRectangle2D.Float(0, 0, getWidth(), getHeight(), arc, arc));
        super.paintChildren(g2);
        g2.dispose();
        Graphics2D g3 = (Graphics2D) g.create();
        smooth(g3);
        g3.setColor(line);
        g3.drawRoundRect(0, 0, getWidth() - 1, getHeight() - 1, arc, arc);
        g3.dispose();
    }
}

// ===== INPUT FIELDS =====
private static void roundBg(Graphics g, JComponent c, Color bg) {
    Graphics2D g2 = (Graphics2D) g.create();
    smooth(g2);
    g2.setColor(bg);
    g2.fillRoundRect(0, 0, c.getWidth(), c.getHeight(), 12, 12);
    g2.dispose();
}

public static void styleField(JTextField f) {
    f.setFont(FONT_REG);
    f.setForeground(NIGHT_NAVY);
    f.setBackground(Color.WHITE);
    f.setCaretColor(DEEP_TEAL);
    f.setOpaque(false);
    f.setBorder(new RoundBorder(BORDER_DARK, 12, 7, 12, 12));
    f.setPreferredSize(new Dimension(120, 36));
    f.addFocusListener(new FocusAdapter() {
        @Override public void focusGained(FocusEvent e) { setBorderColor(f, DEEP_TEAL); }
        @Override public void focusLost(FocusEvent e) { setBorderColor(f, BORDER_DARK); }
    });
```

```
    }

    private static void setBorderColor(JTextField f, Color c) {
        if (f.getBorder() instanceof RoundBorder) ((RoundBorder) f.getBorder()).color = c;
        f.repaint();
    }

    public static JTextField field(String text) {
        JTextField f = new JTextField(text) {
            @Override protected void paintComponent(Graphics g) {
                roundBg(g, this, getBackground());
                super.paintComponent(g);
            }
        };
        styleField(f);
        return f;
    }

    public static JPasswordField passwordField() {
        JPasswordField f = new JPasswordField() {
            @Override protected void paintComponent(Graphics g) {
                roundBg(g, this, getBackground());
                super.paintComponent(g);
            }
        };
        styleField(f);
        return f;
    }

    public static JTextField searchField(String hint) {
        JTextField f = new JTextField() {
            @Override protected void paintComponent(Graphics g) {
                roundBg(g, this, getBackground());
                super.paintComponent(g);
                Graphics2D g2 = (Graphics2D) g.create();
                smooth(g2);
                int cy = getHeight() / 2;
                g2.setColor(MUTED);
                g2.setStroke(new BasicStroke(1.8f, BasicStroke.CAP_ROUND, BasicStroke.JOIN_ROUND));
                g2.drawOval(13, cy - 7, 10, 10);
                g2.drawLine(22, cy + 2, 26, cy + 6);
                if (getText().isEmpty() && !isFocusOwner()) {
                    g2.setFont(FONT_REG);
                    FontMetrics fm = g2.getFontMetrics();
                    g2.drawString(hint, getInsets().left, (getHeight() + fm.getAscent() - fm.getDescent()) / 2);
                }
                g2.dispose();
            }
        };
        styleField(f);
        f.setBorder(new RoundBorder(BORDER_DARK, 12, 7, 36, 12));
        f.setPreferredSize(new Dimension(280, 38));
        return f;
    }

    /** Makes a field display-only (auto IDs, auto owner, client view). */
    public static void lock(JTextField f) {
        f.setEditable(false);
        f.setBackground(Color.decode("#EEF2F7"));
        f.setForeground(STEEL_BLUE);
    }

    public static void readOnly(Component... comps) {
        for (Component c : comps) {
            if (c instanceof JTextField) lock((JTextField) c);
            else if (c instanceof JComboBox) c.setEnabled(false);
        }
    }

    public static JComboBox<String> combo(String... items) {
        JComboBox<String> cb = new JComboBox<String>(items) {
            @Override protected void paintComponent(Graphics g) {
                roundBg(g, this, isEnabled() ? Color.WHITE : Color.decode("#EEF2F7"));
                super.paintComponent(g);
            }
        };
        cb.setUI(new ModernComboUI());
```

```
        cb.setOpaque(false);
        cb.setFont(FONT_REG);
        cb.setForeground(NIGHT_NAVY);
        cb.setBackground(Color.WHITE);
        cb.setBorder(new RoundBorder(BORDER_DARK, 12, 2, 6, 2));
        cb.setPreferredSize(new Dimension(120, 36));
        cb.setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
        cb.setRenderer(new DefaultListCellRenderer() {
            @Override public Component getListCellRendererComponent(JList<?> l, Object v, int i, boolean sel, boolean foc) {
                super.getListCellRendererComponent(l, v, i, sel, foc);
                setBorder(new EmptyBorder(6, 10, 6, 10));
                setFont(FONT_REG);
                setForeground(NIGHT_NAVY);
                setOpaque(true);
                setBackground(sel ? Color.decode("#CCFBF1") : Color.WHITE);
                return this;
            }
        });
        return cb;
    }

    public static class ModernComboUI extends BasicComboBoxUI {
        @Override protected JButton createArrowButton() {
            JButton b = new JButton() {
                @Override public void paint(Graphics g) {
                    Graphics2D g2 = (Graphics2D) g.create();
                    smooth(g2);
                    int cx = getWidth() / 2, cy = getHeight() / 2;
                    g2.setColor(STEEL_BLUE);
                    g2.setStroke(new BasicStroke(2f, BasicStroke.CAP_ROUND, BasicStroke.JOIN_ROUND));
                    g2.drawLine(cx - 4, cy - 2, cx, cy + 2);
                    g2.drawLine(cx, cy + 2, cx + 4, cy - 2);
                    g2.dispose();
                }
            };
            b.setBorder(BorderFactory.createEmptyBorder());
            b.setContentAreaFilled(false);
            b.setFocusPainted(false);
            b.setOpaque(false);
            return b;
        }

        @Override public void paintCurrentValueBackground(Graphics g, Rectangle bounds, boolean hasFocus) { }
    }

    /** Label on top, input below. */
    public static JPanel box(String label, JComponent input) {
        JPanel p = new JPanel(new BorderLayout(0, 5));
        p.setOpaque(false);
        p.add(label(label, FONT_SMALL, STEEL_BLUE), BorderLayout.NORTH);
        p.add(input, BorderLayout.CENTER);
        return p;
    }

    public static JPanel formCard(int cols) {
        RoundPanel p = new RoundPanel(18, CARD_BG);
        p.setLayout(new GridLayout(0, cols, 18, 12));
        p.setBorder(new EmptyBorder(16, 18, 16, 18));
        return p;
    }

    // ===== TABLES =====
    public static DefaultTableModel model(String... cols) {
        return new DefaultTableModel(cols, 0) {
            @Override public boolean isCellEditable(int r, int c) { return false; }
        };
    }

    private static Color statusColor(String s) {
        switch (s) {
            case "Paid": case "Done": case "Completed": case "Healthy": case "Vaccinated":
                return Color.decode("#0F766E");
            case "Pending": case "Partially Vaccinated": case "Under Treatment":
                return Color.decode("#B45309");
            case "Scheduled":
                return Color.decode("#1D4ED8");
            case "Critical": case "Cancelled": case "Not Vaccinated":
```

```
            return Color.decode("#B91C1C");
        default:
            return NIGHT_NAVY;
    }
}

public static JScrollPane styleTable(JTable table) {
    table.setFont(FONT_REG);
    table.setRowHeight(36);
    table.setShowVerticalLines(false);
    table.setShowHorizontalLines(true);
    table.setGridColor(Color.decode("#EEF2F7"));
    table.setIntercellSpacing(new Dimension(0, 1));
    table.setSelectionBackground(Color.decode("#CCFBF1"));
    table.setSelectionForeground(NIGHT_NAVY);
    table.setFillsViewportHeight(true);
    table.setAutoCreateRowSorter(true);
    table.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);

    JTableHeader h = table.getTableHeader();
    h.setReorderingAllowed(false);
    h.setPreferredSize(new Dimension(100, 40));
    h.setBackground(DARK_SLATE);
    h.setDefaultRenderer(new DefaultTableCellRenderer() {
        @Override public Component getTableCellRendererComponent(JTable t, Object v, boolean s, boolean f, int r, int c) {
            super.getTableCellRendererComponent(t, v, false, false, r, c);
            setBackground(DARK_SLATE);
            setForeground(Color.WHITE);
            setFont(FONT_BOLD);
            setBorder(new EmptyBorder(0, 12, 0, 12));
            setHorizontalAlignment(SwingConstants.LEFT);
            return this;
        }
    });
    table.setDefaultRenderer(Object.class, new DefaultTableCellRenderer() {
        @Override public Component getTableCellRendererComponent(JTable t, Object v, boolean sel, boolean f, int r, int c) {
            super.getTableCellRendererComponent(t, v, sel, false, r, c);
            setBorder(new EmptyBorder(0, 12, 0, 12));
            setFont(FONT_REG);
            setHorizontalAlignment(SwingConstants.LEFT);
            if (sel) {
                setBackground(Color.decode("#CCFBF1"));
                setForeground(NIGHT_NAVY);
            } else {
                setBackground(r % 2 == 0 ? Color.WHITE : Color.decode("#F8FAFC"));
                setForeground(statusColor(String.valueOf(v)));
            }
            return this;
        }
    });
    JScrollPane sp = new JScrollPane(table);
    sp.setBorder(BorderFactory.createEmptyBorder());
    sp.getViewport().setBackground(Color.WHITE);
    sp.getVerticalScrollBar().setUnitIncrement(16);
    return sp;
}

public static JPanel tableCard(JTable table) {
    RoundPanel p = new RoundPanel(18, CARD_BG);
    p.setLayout(new BorderLayout());
    p.add(styleTable(table), BorderLayout.CENTER);
    return p;
}

public static JPanel sectionCard(String title, JTable table) {
    RoundPanel p = new RoundPanel(18, CARD_BG);
    p.setLayout(new BorderLayout());
    JLabel l = label(title, FONT_HEADER, NIGHT_NAVY);
    l.setBorder(new EmptyBorder(14, 16, 12, 16));
    p.add(l, BorderLayout.NORTH);
    p.add(styleTable(table), BorderLayout.CENTER);
    return p;
}

@SuppressWarnings({"rawtypes", "unchecked"})
```

```
public static void attachSearch(JTextField box, JTable table) { DocumentListener dl = new DocumentListener() { private void apply() { SwingUtilities.invokeLater(() -> { try { table.clearSelection(); // Clears highlight
before sorter resizes view TableRowSorter sorter = (TableRowSorter) table.getRowSorter(); if (sorter != null) { String t = box.getText().trim(); sorter.setRowFilter(t.isEmpty() ? null : RowFilter.regexFilter("(?i)" +
Pattern.quote(t))); } } catch (Exception ignored) { // Prevents console warnings during rapid typing } }); } public void insertUpdate(DocumentEvent e) { apply(); } public void removeUpdate(DocumentEvent e)
{ apply(); } public void changedUpdate(DocumentEvent e) { apply(); } }; box.getDocument().addDocumentListener(dl); }

// ===== MESSAGES =====
public static void showError(Component parent, Exception ex) {    ex.printStackTrace();
    String msg = ex.getMessage() == null ? ex.toString() : ex.getMessage();
    if (ex instanceof java.sql.SQLException && ((java.sql.SQLException) ex).getErrorCode() == 1062)
        msg = "That record already exists.";
    else if (ex instanceof java.sql.SQLException)
        msg = "Database problem: " + msg + "\nMake sure MySQL is running.";
    JOptionPane.showMessageDialog(parent, msg, "PetPulse", JOptionPane.ERROR_MESSAGE);
}

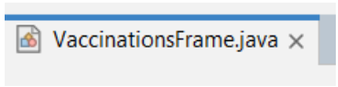
/** Small slide-in notification at the bottom-right of the window. */
public static void toast(Component any, String msg, boolean ok) {
    Window w = any instanceof Window ? (Window) any : SwingUtilities.getWindowAncestor(any);
    if (!(w instanceof RootPaneContainer)) return;
    new Toast(((RootPaneContainer) w).getLayeredPane(), msg, ok ? DEEP_TEAL : ARCHIVE_RED);
}

private static class Toast extends JComponent {
    private final String msg;
    private final Color accent;
    private float alpha = 1f;

    Toast(JLayeredPane lp, String msg, Color accent) {
        this.msg = msg;
        this.accent = accent;
        FontMetrics fm = getFontMetrics(FONT_BOLD);
        int w = Math.min(fm.stringWidth(msg) + 70, Math.max(300, lp.getWidth() - 40)), h = 48;
        setSize(w, h);
        final int x = lp.getWidth() - w - 28;
        final int endY = lp.getHeight() - h - 28;
        final int startY = lp.getHeight() + 10;
        setLocation(x, startY);
        lp.add(this, JLayeredPane.POPUP_LAYER);
        final long t0 = System.currentTimeMillis();
        Timer t = new Timer(16, null);
        t.addActionListener(e -> {
            long dt = System.currentTimeMillis() - t0;
            if (dt < 260) {
                float p = dt / 260f;
                p = 1 - (1 - p) * (1 - p);
                setLocation(x, (int) (startY + (endY - startY) * p));
            } else if (dt < 2900) {
                setLocation(x, endY);
            } else if (dt < 3200) {
                alpha = 1 - (dt - 2900) / 300f;
                repaint();
            } else {
                t.stop();
                lp.remove(this);
                lp.repaint();
            }
        });
        t.start();
    }

    @Override protected void paintComponent(Graphics g) {
        Graphics2D g2 = (Graphics2D) g.create();
        smooth(g2);
        g2.setComposite(AlphaComposite.SrcOver.derive(Math.max(0f, Math.min(1f, alpha))));
        g2.setColor(DARK_SLATE);
        g2.fillRoundRect(0, 0, getWidth(), getHeight(), 16, 16);
        g2.setColor(accent);
        g2.fillRoundRect(0, 0, 8, getHeight(), 8, 8);
        g2.fillOval(24, getHeight() / 2 - 5, 10, 10);
        g2.setColor(Color.WHITE);
        g2.setFont(FONT_BOLD);
        FontMetrics fm = g2.getFontMetrics();
        g2.drawString(msg, 46, (getHeight() + fm.getAscent() - fm.getDescent()) / 2);
        g2.dispose();
    }
}
```

}




```
package petpulse;

import java.awt.*; import java.sql.*; import java.time.LocalDate; import javax.swing.*; import javax.swing.table.DefaultTableModel;

/** Vaccination Booking & Management Module. */ public class VaccinationsFrame extends BaseFrame { private final JTextField txtId = UITheme.field("(auto)"), txtGiven =
UITheme.field(LocalDate.now().toString()), txtDue = UITheme.field(LocalDate.now().plusYears(1).toString()), txtDosage = UITheme.field("1 ml"); private final JTextField txtSearch = UITheme.searchField("Search
vaccinations..."); private final JComboBox cbPet = UITheme.combo(); private final JComboBox cbVaccine = UITheme.combo("Rabies", "DHPP", "Bordetella", "Parvovirus", "Distemper", "FVRCP"); private final
JComboBox cbVet = UITheme.combo("Dr. Grey", "Dr. House"); private final JComboBox cbStatus = UITheme.combo("Done", "Pending"); private final DefaultTableModel model = UITheme.model("Vaccination
ID", "Pet Name", "Vaccine", "Date Given", "Next Due Date", "Dosage", "Veterinarian", "Status"); private final JTable table = new JTable(model);

public VaccinationsFrame() {
    super("Vaccinations", "Vaccinations", "Record immunizations and schedule booster visits");
    boolean admin = Session.isAdmin();
    UITheme.lock(txtId);

    Db.bindPets(cbPet, null);

    JPanel form = UITheme.formCard(4);
    form.add(UITheme.box("Vaccination ID", txtId));
    form.add(UITheme.box("Pet Name / ID", cbPet));
    form.add(UITheme.box("Vaccine Name", cbVaccine));
    form.add(UITheme.box("Dosage", txtDosage));
    form.add(UITheme.box("Date Given (YYYY-MM-DD)", txtGiven));
    form.add(UITheme.box("Next Due Date (YYYY-MM-DD)", txtDue));
    form.add(UITheme.box("Veterinarian", cbVet));
    form.add(UITheme.box("Status", cbStatus));

    JButton btnAdd = UITheme.createBtn(admin ? "Add" : "Book Vaccination", UITheme.BRIGHT_TEAL);
    JButton btnUpd = UITheme.createBtn("Update", UITheme.STEEL_BLUE);
    JButton btnArc = UITheme.createBtn("Archive", UITheme.ARCHIVE_RED);
    JButton btnClr = UITheme.createBtn("Clear", UITheme.STEEL_BLUE);

    JPanel btns = new JPanel(new FlowLayout(FlowLayout.LEFT, 10, 0));
    btns.setOpaque(false);
    btns.add(btnAdd);
    btns.add(btnUpd);
    btns.add(btnArc);
    btns.add(btnClr);

    JPanel bar = new JPanel(new BorderLayout());
    bar.setOpaque(false);
    bar.add(btns, BorderLayout.WEST);
    bar.add(txtSearch, BorderLayout.EAST);

    JPanel top = new JPanel(new BorderLayout(0, 12));
    top.setOpaque(false);
    top.add(form, BorderLayout.NORTH);
    top.add(bar, BorderLayout.SOUTH);

    JPanel root = new JPanel(new BorderLayout(0, 14));
    root.setOpaque(false);
    root.add(top, BorderLayout.NORTH);
    root.add(UITheme.tableCard(table), BorderLayout.CENTER);
    setBody(root);

    UITheme.attachSearch(txtSearch, table);
    if (!admin) cbStatus.setEnabled(false);

    table.getSelectionModel().addListSelectionListener(e -> {
        if (e.getValueIsAdjusting()) return;
        int r = table.getSelectedRow();
        if (r < 0 || r >= table.getRowCount()) return;
        try {
            r = table.convertRowIndexToModel(r);
            txtId.setText(v(r, 0));
            Db.selectPet(cbPet, v(r, 1));
            cbVaccine.setSelectedItem(v(r, 2));
            txtGiven.setText(v(r, 3));
            txtDue.setText(v(r, 4));
            txtDosage.setText(v(r, 5));
```

```
            cbVet.setSelectedItem(v(r, 6));
            cbStatus.setSelectedItem(v(r, 7));
        } catch (Exception ex) { }
    });

    btnClr.addActionListener(e -> clearForm());
    btnAdd.addActionListener(e -> addRecord());
    btnUpd.addActionListener(e -> updateRecord());
    btnArc.addActionListener(e -> archiveRecord());
    loadData();
}

private String v(int r, int c) {
    Object val = model.getValueAt(r, c);
    return val == null ? "" : String.valueOf(val);
}

private void clearForm() {
    txtId.setText("(auto)");
    txtGiven.setText(LocalDate.now().toString());
    txtDue.setText(LocalDate.now().plusYears(1).toString());
    txtDosage.setText("1 ml");
    cbVaccine.setSelectedIndex(0);
    cbVet.setSelectedIndex(0);
    cbStatus.setSelectedIndex(0);
    if (cbPet.getItemCount() > 0) cbPet.setSelectedIndex(0);
    table.clearSelection();
}

private String extractPetName(Object item) {
    if (item == null) return "";
    String s = item.toString().trim();
    return s.contains(" - ") ? s.split(" - ", 2)[1].trim() : s;
}

private String check() {
    Object selectedPet = cbPet.getSelectedItem();
    if (selectedPet == null || selectedPet.toString().trim().isEmpty()) {
        return "Register or select a pet first (Pet Information).";
    }
    return Validation.firstError(
        Validation.date(txtGiven.getText(), "Date given"),
        Validation.date(txtDue.getText(), "Next due date"),
        Validation.dateOrder(txtGiven.getText(), txtDue.getText()),
        Validation.required(txtDosage.getText(), "Dosage"));
}

private void addRecord() {
    String err = check();
    if (err != null) { UITheme.toast(this, err, false); return; }
    String petName = extractPetName(cbPet.getSelectedItem());
    String vaccine = cbVaccine.getSelectedItem().toString();
    String vet = cbVet.getSelectedItem().toString();
    String status = Session.isAdmin() ? cbStatus.getSelectedItem().toString() : "Pending";

    try (Connection c = DBConnection.getConnection()) {
        String id = Db.nextId(c, "Vaccinations", "VAC-");
        try (PreparedStatement ps = c.prepareStatement(
                "INSERT INTO vaccinations (vaccination_id, pet_name, vaccine_name, date_given, next_due_date, dosage, veterinarian, status) VALUES (?,?,?,?,?,?,?,?)")) {
            ps.setString(1, id);
            ps.setString(2, petName);
            ps.setString(3, vaccine);
            ps.setString(4, txtGiven.getText().trim());
            ps.setString(5, txtDue.getText().trim());
            ps.setString(6, txtDosage.getText().trim());
            ps.setString(7, vet);
            ps.setString(8, status);
            ps.executeUpdate();
        }
        DBConnection.log("Vaccinations", "Add", id + " - " + petName + " (" + vaccine + ")");
        UITheme.toast(this, "Vaccination booked: " + id, true);
        clearForm();
        loadData();
    } catch (Exception ex) { UITheme.showError(this, ex); }
}
```

```
private void updateRecord() {
    if ("(auto)".equals(txtId.getText())) { UITheme.toast(this, "Select a record from the table first.", false); return; }
    String err = check();
    if (err != null) { UITheme.toast(this, err, false); return; }

    String petName = extractPetName(cbPet.getSelectedItem());
    String vaccine = cbVaccine.getSelectedItem().toString();
    String vet = cbVet.getSelectedItem().toString();
    String status = cbStatus.getSelectedItem().toString();

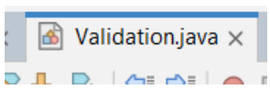
    try (Connection c = DBConnection.getConnection();
         PreparedStatement ps = c.prepareStatement(
                 "UPDATE vaccinations SET pet_name=?, vaccine_name=?, date_given=?, next_due_date=?, dosage=?, veterinarian=?, status=? WHERE vaccination_id=?")) {
        ps.setString(1, petName);
        ps.setString(2, vaccine);
        ps.setString(3, txtGiven.getText().trim());
        ps.setString(4, txtDue.getText().trim());
        ps.setString(5, txtDosage.getText().trim());
        ps.setString(6, vet);
        ps.setString(7, status);
        ps.setString(8, txtId.getText());
        ps.executeUpdate();

        DBConnection.log("Vaccinations", "Update", txtId.getText() + " updated");
        UITheme.toast(this, "Vaccination record updated.", true);
        clearForm();
        loadData();
    } catch (Exception ex) { UITheme.showError(this, ex); }
}

private void archiveRecord() {
    if ("(auto)".equals(txtId.getText())) { UITheme.toast(this, "Select a record from the table first.", false); return; }
    if (JOptionPane.showConfirmDialog(this, "Move " + txtId.getText() + " to the archive?", "Archive Vaccination", JOptionPane.YES_NO_OPTION) != JOptionPane.YES_OPTION) return;
    try {
        Db.archiveRecord("Vaccinations", txtId.getText(), extractPetName(cbPet.getSelectedItem()) + " (" + cbVaccine.getSelectedItem() + ")");
        DBConnection.log("Vaccinations", "Archive", txtId.getText() + " archived");
        UITheme.toast(this, "Vaccination moved to archive.", true);
        clearForm();
        loadData();
    } catch (Exception ex) { UITheme.showError(this, ex); }
}

private void loadData() {
    try {
        if (Session.isAdmin()) {
            Db.fill(model, "SELECT vaccination_id, pet_name, vaccine_name, date_given, next_due_date, dosage, veterinarian, status FROM vaccinations ORDER BY vaccination_id", 8);
        } else {
            String owner = Session.getFullName();
            model.setRowCount(0);
            try (Connection c = DBConnection.getConnection();
                 PreparedStatement ps = c.prepareStatement(
                    "SELECT v.vaccination_id, v.pet_name, v.vaccine_name, v.date_given, v.next_due_date, v.dosage, v.veterinarian, v.status " +
                    "FROM vaccinations v JOIN pets p ON v.pet_name = p.pet_name " +
                    "WHERE p.owner_name=? ORDER BY v.vaccination_id")) {
                ps.setString(1, owner);
                try (ResultSet rs = ps.executeQuery()) {
                    while (rs.next()) {
                        Object[] row = new Object[8];
                        for (int i = 0; i < 8; i++) {
                            String val = rs.getString(i + 1);
                            row[i] = val == null ? "" : val;
                        }
                        model.addRow(row);
                    }
                }
            }
        }
    } catch (Exception e) {
        e.printStackTrace();
        UITheme.toast(this, "Could not load vaccinations. Is MySQL running?", false);
    }
}


}
```




```
package petpulse;

import java.math.BigDecimal; import java.time.LocalDate;

/** Input checks. Each method returns null when valid, or an error message. */ public class Validation {

public static String required(String value, String label) {
    return (value == null || value.trim().isEmpty()) ? label + " is required." : null;
}

public static String age(String v) {
    try {
        int a = Integer.parseInt(v.trim());
        return (a < 0 || a > 40) ? "Age must be between 0 and 40." : null;
    } catch (Exception e) {
        return "Age must be a whole number.";
    }
}

public static String phone(String v) {
    String d = v == null ? "" : v.replaceAll("[\\s-]", "");
    return d.matches("\\+?\\d{7,13}") ? null : "Contact number must have 7 to 13 digits.";
}

public static String date(String v, String label) {
    try {
        LocalDate.parse(v.trim());
        return null;
    } catch (Exception e) {
        return label + " must be YYYY-MM-DD (example: 2026-10-15).";
    }
}

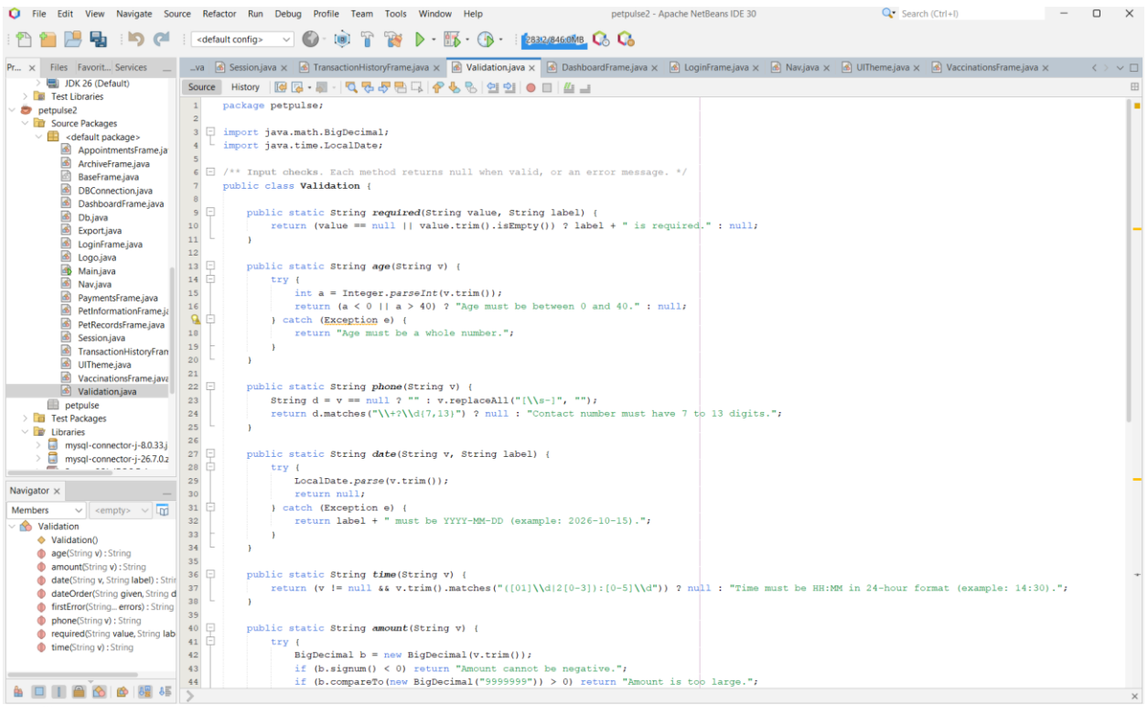
public static String time(String v) {
    return (v != null && v.trim().matches("([01]\\d|2[0-3]):[0-5]\\d")) ? null : "Time must be HH:MM in 24-hour format (example: 14:30).";
}

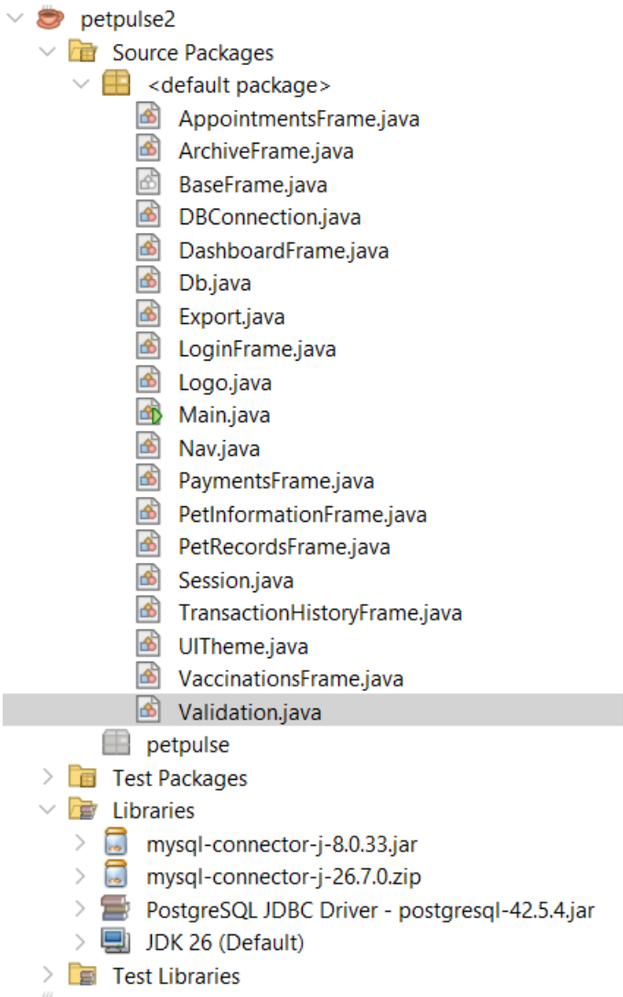
public static String amount(String v) {
    try {
        BigDecimal b = new BigDecimal(v.trim());
        if (b.signum() < 0) return "Amount cannot be negative.";
        if (b.compareTo(new BigDecimal("9999999")) > 0) return "Amount is too large.";
        return null;
    } catch (Exception e) {
        return "Amount must be a valid number (example: 500.00).";
    }
}

/** Next due date must not be before the date given. */
public static String dateOrder(String given, String due) {
    try {
        if (LocalDate.parse(due.trim()).isBefore(LocalDate.parse(given.trim())))
            return "Next due date cannot be before the date given.";
    } catch (Exception ignored) { }
    return null;
}

public static String firstError(String... errors) {
    for (String e : errors) if (e != null) return e;
    return null;
}


}
```


File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help
petpulse2 - Apache NetBeans IDE 30
Search (Ctrl+I)
<default config>
Files Favorit... Services
JDK 26 (Default)
Test Libraries
petpulse2
Source Packages
<default package>
AppointmentsFrame.ja
ArchiveFrame.java
BaseFrame.java
DBConnection.java
DashboardFrame.java
Db.java
Export.java
LoginFrame.java
Logo.java
Main.java
Nav.java
PaymentsFrame.java
PetInformationFrame.ja
PetRecordsFrame.java
Session.java
TransactionHistoryFran
UITheme.java
VaccinationsFrame.java
Validation.java
petpulse
Test Packages
Libraries
mysql-connector-j-8.0.33.j
mysql-connector-j-26.7.0.z
Navigator
Members
Validation
Validation()
age(String v) : String
amount(String v) : String
date(String v, String label) : Stri
dateOrder(String given, String d
firstError(String... errors) : String
phone(String v) : String
required(String value, String lab
time(String v) : String
...va
Session.java
TransactionHistoryFrame.java
Validation.java
DashboardFrame.java
LoginFrame.java
Nav.java
UITheme.java
VaccinationsFrame.java
Source
History
package petpulse;
import java.math.BigDecimal;
import java.time.LocalDate;
/** Input checks. Each method returns null when valid, or an error message. */
public class Validation {
public static String required(String value, String label) {
return (value == null || value.trim().isEmpty()) ? label + " is required." : null;
}
public static String age(String v) {
try {
int a = Integer.parseInt(v.trim());
return (a < 0 || a > 40) ? "Age must be between 0 and 40." : null;
} catch (Exception e) {
return "Age must be a whole number.";
}
}
public static String phone(String v) {
String d = v == null ? "" : v.replaceAll("[\\s-]", "");
return d.matches("\\+?\\d{7,13}") ? null : "Contact number must have 7 to 13 digits.";
}
public static String date(String v, String label) {
try {
LocalDate.parse(v.trim());
return null;
} catch (Exception e) {
return label + " must be YYYY-MM-DD (example: 2026-10-15).";
}
}
public static String time(String v) {
return (v != null && v.trim().matches("([01]\\d|2[0-3]):[0-5]\\d")) ? null : "Time must be HH:MM in 24-hour format (example: 14:30).";
}
public static String amount(String v) {
try {
BigDecimal b = new BigDecimal(v.trim());
if (b.signum() < 0) return "Amount cannot be negative.";
if (b.compareTo(new BigDecimal("9999999")) > 0) return "Amount is too large.";


petpulse2
Source Packages
<default package>
AppointmentsFrame.java
ArchiveFrame.java
BaseFrame.java
DBConnection.java
DashboardFrame.java
Db.java
Export.java
LoginFrame.java
Logo.java
Main.java
Nav.java
PaymentsFrame.java
PetInformationFrame.java
PetRecordsFrame.java
Session.java
TransactionHistoryFrame.java
UITheme.java
VaccinationsFrame.java
Validation.java
petpulse
Test Packages
Libraries
mysql-connector-j-8.0.33.jar
mysql-connector-j-26.7.0.zip
PostgreSQL JDBC Driver - postgresql-42.5.4.jar
JDK 26 (Default)
Test Libraries